HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

o0o

BÁO CÁO BÀI TẬP VỀ NHÀ CHƯƠNG 03

Chủ đề: BÀI TOÁN DUYỆT

Học phần: Cấu trúc Dữ liệu và Giải thuật (mã HP: INT1306)

Yêu cầu lập trình trên 3 ngôn ngữ: Python, C++, Julia

Thông tin sinh viên	Sinh viên tự điền
Họ và tên	Phạm Đỗ Duy Thanh
Mã số sinh viên (MSSV)	N24DCDT086
Lớp / Nhóm học phần	D24CQDT01-N
Bộ dữ liệu chuẩn	dùng chung cả lớp — xem trang sau, không được tự đổi

Quy định làm bài và nộp báo cáo (bắt buộc)

1. Không được để trống các khung bên dưới. Phải tự tay trình bày đầy đủ lời giải, vẽ bảng lần vết, viết mã nguồn và chèn ảnh chụp màn hình kiểm thử.

2. Mã nguồn phải cài đặt trên đủ ba ngôn ngữ của bài giảng: Python, C++ và Julia (chú ý Julia đánh chỉ số từ 1).

3. Ở các mục yêu cầu hình ảnh, chụp màn hình IDE (VS Code, Jupyter, Terminal) có hiển thị mã nguồn và cửa sổ kết quả chạy thực nghiệm rồi dán trực tiếp vào khung tương ứng.

4. Mọi con số báo cáo phải là số chương trình in ra, không được chép của bạn. Bộ dữ liệu chuẩn giống nhau cho cả lớp nên đáp số đúng chỉ có một; điều được chấm là lần vết, lập luận và mã nguồn của chính sinh viên.

5. Nộp file .docx hoặc .pdf, đặt tên theo mẫu `BTVN_Chuong03_[MSSV]_[HoTen]`.

Phần	Nội dung	Điểm
1	Câu hỏi lý thuyết và chứng minh học thuật (ba câu)	3,0
2	Quay lui giải bài toán n quân hậu, trình bày theo bảy bước chuẩn	**4,0**
3	Lần vết ba thuật toán sinh kế tiếp	1,5
4	Ba bài toán ứng dụng trên ba ngôn ngữ	1,5
	TỔNG	**10,0**

Hạn nộp: trước 23 giờ 59 ngày thứ Tư 09/09/2026, nộp lên Google Classroom. Bài nộp muộn trừ 1,0 điểm mỗi ngày.

Bộ dữ liệu chuẩn dùng chung cả lớp

Mọi sinh viên làm trên đúng bộ dữ liệu dưới đây. Nhập sai một con số là toàn bộ kết quả sai theo, nên hãy đối chiếu kỹ trước khi chạy chương trình.

A. Ba cấu hình xuất phát cho phần 3

Loại cấu hình	Cấu hình xuất phát	Tham số
Xâu nhị phân	`10110`	n = 5
Tổ hợp chập k của n	`(1, 3, 6, 8)`	n = 8, k = 4
Hoán vị	`(3, 1, 5, 4, 2)`	n = 5

B. Bàn cờ cho phần 2

Lần vết bằng tay trên bàn cờ **n = 4**. Chạy chương trình cho **n = 5, 6 và 8**.

C. Bộ đồ vật cho bài 4.1 — bài toán cái túi

Đồ vật	A	B	C	D	E	F
Trọng lượng w	4	7	3	3	3	10
Giá trị v	19	25	7	8	9	21

Sức chứa của túi: **W = 14**. Mỗi đồ vật chỉ được lấy nguyên hoặc bỏ hẳn, không được cắt nhỏ.

D. Ma trận chi phí cho bài 4.2 — bài toán người du lịch

Từ \ Đến	0	1	2	3	4
0	0	10	9	22	23
1	10	0	16	19	14
2	9	16	0	7	28
3	22	19	7	0	8
4	23	14	28	8	0

Ma trận chi phí giữa năm thành phố. Ma trận đối xứng nên đi chiều nào cũng cùng chi phí.

Năm thành phố đánh số từ 0 đến 4. Ma trận đối xứng, xuất phát và kết thúc ở thành phố 0.

E. Đề Sudoku cho bài 4.3

					6			
	8	1		5		4		
		9			7		5	8
				9				
		6			1			7
7	4					2		1
			1	3				4
				2	9			
9					8		6	

Ô tô nền và in đậm là số cho trước. Ô trắng là ô phải điền.

Đề có **56 ô trống** và có đúng một lời giải. Hãy đếm lại số ô trống ngay khi nhập xong để chắc chắn không gõ nhầm — nhập sai một ô là chương trình có thể chạy mãi không ra, hoặc ra một lời giải khác.

> **Cách nhập lưới vào chương trình**
>
> Ghi mỗi hàng thành một dãy chín chữ số, ô trống ghi số 0. Hàng đầu của đề là `000006000`. Nhập đủ chín hàng theo thứ tự từ trên xuống.
>
> Kiểm tra ngay sau khi nhập: đếm số 0 trong cả lưới phải ra đúng 56, và mỗi hàng, mỗi cột, mỗi khối 3 × 3 không được có hai chữ số giống nhau.

Phần 1. Câu hỏi lý thuyết và chứng minh học thuật (3,0 điểm)

Câu 1.1 (1,0 điểm) — Không gian trạng thái và bốn dạng bài toán duyệt

a) Định nghĩa không gian trạng thái và cây không gian trạng thái. Với bài toán liệt kê xâu nhị phân độ dài n, cây có bao nhiêu nút và bao nhiêu lá? Chứng minh hai công thức đó.

b) Nêu bốn dạng bài toán duyệt và cho biết mỗi dạng kết thúc thuật toán theo cách nào. Vì sao riêng dạng tìm một lời giải cần hàm đệ quy trả về giá trị lô-gic, còn ba dạng kia thì không?

a) Định nghĩa và tính số nút, số lá của bài toán liệt kê xâu nhị phân độ dài n

Không gian trạng thái (State Space):

Không gian trạng thái của một bài toán tìm kiếm hoặc duyệt là tập hợp tất cả các trạng thái có thể xuất hiện trong quá trình tìm kiếm lời giải, bao gồm trạng thái ban đầu, các trạng thái trung gian (nghiệm bộ phận) và các trạng thái kết thúc (nghiệm hoàn chỉnh hoặc trạng thái bế tắc).

Cây không gian trạng thái (State Space Tree):

Cây không gian trạng thái là một cấu trúc cây có gốc dùng để biểu diễn quá trình tìm kiếm lời giải.

- Nút gốc (mức 0): đại diện cho trạng thái ban đầu khi chưa chọn phần tử nào, tức nghiệm rỗng.
- Nút ở mức k (1 <= k <= n): đại diện cho nghiệm bộ phận (x1, x2, ..., xk) sau khi đã gán giá trị cho k thành phần đầu tiên.
- Cạnh nối từ mức k-1 sang mức k: tương ứng với một quyết định gán một giá trị khả dĩ cho biến xk.
- Nút lá ở mức n: đại diện cho một cấu hình đầy đủ, tức một xâu nhị phân hoàn chỉnh.

Đối với bài toán liệt kê xâu nhị phân độ dài n, tại mỗi vị trí có 2 lựa chọn là 0 hoặc 1.

Số nút lá:

Ở mỗi vị trí của xâu có 2 khả năng. Vì xâu có n vị trí nên số xâu nhị phân khác nhau là:

L = 2^n

Do đó số nút lá của cây là:

L = 2^n

Số nút của cây:

Ở mức 0 có 2^0 = 1 nút.

Ở mức 1 có 2^1 = 2 nút.

Ở mức 2 có 2^2 = 4 nút.

...

Ở mức n có 2^n nút.

Vì vậy tổng số nút là:

N = 2^0 + 2^1 + 2^2 + ... + 2^n

Đây là tổng của một cấp số nhân có công bội q = 2, nên:

N = 2^(n+1) - 1

Vậy:

Số nút = 2^(n+1) - 1

Số lá = 2^n

Chứng minh số lá:

Các nút lá nằm ở mức n và tương ứng với tất cả các xâu nhị phân có độ dài n.
Mỗi vị trí có 2 khả năng là 0 hoặc 1. Do có n vị trí nên theo quy tắc nhân:
L = 2 x 2 x ... x 2 = 2^n
Vậy số lá là 2^n.
Chứng minh số nút:
Tại mức k có đúng 2^k nút vì mỗi nút ở mức k-1 sinh ra 2 nút ở mức k.
Tổng số nút từ mức 0 đến mức n là:
N = 2^0 + 2^1 + 2^2 + ... + 2^n
Nhân hai vế với 2:
2N = 2^1 + 2^2 + ... + 2^n + 2^(n+1)
Lấy phương trình thứ hai trừ phương trình thứ nhất:
2N - N = 2^(n+1) - 2^0
Suy ra:
N = 2^(n+1) - 1
Vậy công thức số nút được chứng minh.
Kết luận:
N = 2^(n+1) - 1
L = 2^n

b) Bốn dạng bài toán duyệt và cơ chế kết thúc

Có bốn dạng bài toán duyệt thường gặp:

1. Tìm một lời giải

Mục tiêu là tìm một nghiệm bất kỳ thỏa mãn các điều kiện của bài toán.
Khi tìm được một lời giải, thuật toán có thể kết thúc ngay mà không cần xét các nhánh còn lại.
Hàm đệ quy thường trả về kiểu Boolean:
True: đã tìm được lời giải.
False: nhánh hiện tại không tìm được lời giải.

2. Tìm tất cả lời giải

Mục tiêu là tìm và liệt kê toàn bộ các nghiệm thỏa mãn điều kiện của bài toán.
Khi tìm được một lời giải, thuật toán không được dừng mà phải tiếp tục quay lui để xét các nhánh còn lại.
Thuật toán kết thúc khi các nhánh cần thiết trong không gian trạng thái đã được duyệt xong.

3. Đếm số lời giải

Mục tiêu là xác định tổng số nghiệm thỏa mãn điều kiện.
Mỗi khi tìm được một lời giải, chương trình tăng biến đếm:
count = count + 1
Sau đó tiếp tục quay lui để xét các nhánh khác.

Thuật toán kết thúc khi toàn bộ không gian trạng thái cần xét đã được duyệt xong.

4. Tối ưu hóa

Mục tiêu là tìm lời giải tốt nhất theo một tiêu chí, chẳng hạn giá trị lớn nhất, nhỏ nhất hoặc chi phí thấp nhất.

Khi tìm được một lời giải, chương trình cập nhật lời giải tốt nhất hiện tại:

Best = current

Sau đó tiếp tục tìm kiếm các khả năng tốt hơn.

Nếu sử dụng kỹ thuật nhánh cận, những nhánh chắc chắn không thể tạo ra lời giải tốt hơn Best có thể được cắt bỏ.

Thuật toán kết thúc khi không còn nhánh nào cần xét có khả năng cải thiện lời giải tốt nhất.

Giải thích vì sao riêng dạng "tìm một lời giải" cần hàm đệ quy trả về giá trị lô-gic

Trong bài toán tìm một lời giải, mục tiêu là tìm được một nghiệm bất kỳ. Vì vậy ngay khi tìm thấy nghiệm, toàn bộ quá trình tìm kiếm phải dừng lại.

Do thuật toán quay lui sử dụng nhiều lời gọi đệ quy lồng nhau, khi một lời gọi ở tầng sâu tìm thấy nghiệm, cần có một tín hiệu để thông báo ngược lên các tầng gọi trước đó rằng đã tìm thấy nghiệm và phải dừng tìm kiếm.

Giá trị Boolean được sử dụng làm tín hiệu này:

True: đã tìm thấy lời giải.

False: chưa tìm thấy lời giải trong nhánh đang xét.

Ví dụ:

```
if Try(i) == True:
return True
```

Khi tầng con trả về True, tầng cha cũng trả về True ngay mà không tiếp tục thử các nhánh khác. Giá trị True tiếp tục được truyền ngược lên cho đến lời gọi ban đầu, nhờ đó toàn bộ quá trình tìm kiếm được dừng sớm.

Đối với ba dạng còn lại:

- Tìm tất cả lời giải: sau khi tìm được một nghiệm vẫn phải tiếp tục tìm các nghiệm khác.
- Đếm số lời giải: sau khi tìm được một nghiệm phải tăng biến đếm và tiếp tục tìm các nghiệm còn lại.
- Tối ưu hóa: sau khi tìm được một nghiệm phải cập nhật lời giải tốt nhất và tiếp tục tìm kiếm nếu vẫn còn khả năng có nghiệm tốt hơn. Nếu dùng nhánh cận thì chỉ những nhánh không còn khả năng cải thiện kết quả mới được cắt bỏ.

Vì vậy, dạng "tìm một lời giải" cần giá trị Boolean để truyền tín hiệu dừng từ tầng đệ quy sâu lên các tầng phía trên. Ba dạng còn lại không cần Boolean với mục đích truyền tín hiệu dừng toàn bộ như vậy.

Kết luận:

Tìm một lời giải -> dùng Boolean để báo hiệu đã tìm thấy nghiệm và dừng sớm.
Tìm tất cả lời giải -> tìm được nghiệm vẫn tiếp tục duyệt.
Đếm số lời giải -> tìm được nghiệm thì tăng biến đếm và tiếp tục duyệt.
Tối ưu hóa -> tìm được nghiệm thì cập nhật nghiệm tốt nhất và tiếp tục tìm kiếm hoặc cắt nhánh bằng điều kiện cận.

Câu 1.2 (1,0 điểm) — Ba thành phần bắt buộc của quay lui

Nêu ba thành phần bắt buộc của một thuật toán quay lui. Với mỗi thành phần, cho biết bỏ sót nó thì chương trình hỏng theo kiểu nào — nêu rõ triệu chứng quan sát được khi chạy, chứ không chỉ nói "sai". Minh hoạ bằng chính bài toán n quân hậu ở phần 2.

BA THÀNH PHẦN BẮT BUỘC CỦA THUẬT TOÁN QUAY LUI

Ba thành phần bắt buộc của một thuật toán quay lui kinh điển gồm:

1. Điều kiện dừng (Base case / Stopping condition)
2. Điều kiện chấp nhận được (Kiểm tra ràng buộc / Cắt tỉa)
3. Khôi phục trạng thái (Backtrack / Undo)
4. ĐIỀU KIỆN DỪNG (BASE CASE / STOPPING CONDITION)

Bản chất:

Điều kiện dừng là điểm kết thúc của quá trình đệ quy khi nghiệm bộ phận đã được xây dựng hoàn chỉnh đến độ dài n, tức đã đến nút lá của cây không gian trạng thái. Khi đó chương trình ghi nhận hoặc in nghiệm và kết thúc nhánh đang xét.

Triệu chứng khi bỏ sót:

- Chương trình không biết khi nào một cấu hình đã hoàn tất.
- Hàm đệ quy tiếp tục gọi đến tầng n+1, n+2, ... vượt quá giới hạn của bài toán.
- Có thể xảy ra tràn ngăn xếp đệ quy (Stack Overflow).
- Trong C++ có thể xảy ra lỗi truy cập bộ nhớ ngoài phạm vi như Segmentation fault.
- Trong Python có thể xuất hiện lỗi RecursionError: maximum recursion depth exceeded.
- Nếu truy cập mảng bằng chỉ số vượt giới hạn, chương trình có thể phát sinh lỗi IndexError hoặc lỗi truy cập ngoài phạm vi.
- Chương trình có thể không in được nghiệm hoàn chỉnh nào.

Minh họa với bài toán n quân hậu:

Khi đã đặt xong quân hậu ở hàng n, lời gọi Try(n + 1) phải kích hoạt điều kiện dừng.

Ví dụ:

if i > n:

in nghiệm

return

Khi đó chương trình biết rằng đã đặt đủ n quân hậu và có thể ghi nhận một nghiệm.

Nếu bỏ qua điều kiện dừng, chương trình tiếp tục gọi Try(n + 1), sau đó tiếp tục thử đặt quân hậu ở hàng n + 1. Khi đó chỉ số hàng vượt quá giới hạn bài toán, có thể dẫn đến lỗi truy cập mảng hoặc tràn ngăn xếp.

2. ĐIỀU KIỆN CHẤP NHẬN ĐƯỢC (BOUNDING CONDITION / PRUNING)

Bản chất:

Điều kiện chấp nhận được là bước kiểm tra xem việc gán giá trị v cho biến xi ở bước hiện tại có vi phạm các ràng buộc của bài toán hay không.

Nếu trạng thái không hợp lệ thì nhánh đó được cắt tỉa ngay lập tức và không tiếp tục đệ quy xuống các mức sâu hơn.

Triệu chứng khi bỏ sót:

- Thuật toán mất khả năng cắt tỉa các nhánh chắc chắn không thể tạo thành nghiệm hợp lệ.
- Số trạng thái phải xét tăng lên rất nhiều.
- Thuật toán có thể trở thành dạng duyệt vét cạn.
- Với bài toán quân hậu, chương trình có thể sinh ra các cấu hình có quân hậu cùng cột hoặc cùng đường chéo.
- Thời gian chạy tăng đáng kể khi n lớn.

Minh họa với bài toán n quân hậu:
Khi đặt quân hậu tại hàng i, cột j, chỉ được chấp nhận nếu:
col[j] == false
d1[i - j + n] == false
d2[i + j] == false
Điều này có nghĩa là:

- Cột j chưa có quân hậu.
- Đường chéo thứ nhất chưa có quân hậu.
- Đường chéo thứ hai chưa có quân hậu.

Chỉ khi cả ba điều kiện đều đúng thì mới được đặt quân hậu vào ô đang xét.
Nếu bỏ qua điều kiện kiểm tra này và đặt quân hậu vào mọi cột từ 1 đến n, chương trình sẽ tạo ra nhiều cấu hình vi phạm luật.
Ví dụ với n = 4, cấu hình:
(1, 1, 1, 1)
có cả bốn quân hậu nằm cùng một cột nên không phải nghiệm.
Cấu hình:
(1, 2, 3, 4)
cũng không hợp lệ vì các quân hậu nằm trên cùng một đường chéo.

3. KHÔI PHỤC TRẠNG THÁI (BACKTRACKING / STATE RESTORATION / UNDO)

Bản chất:
Sau khi một nhánh đệ quy con kết thúc, thuật toán phải hoàn tác các thay đổi đã thực hiện trên cấu trúc dữ liệu trạng thái. Mục đích là đưa trạng thái trở về đúng như trước khi thử nhánh hiện tại để có thể thử các lựa chọn khác.
Đây chính là thao tác quay lui (Backtracking).
Triệu chứng khi bỏ sót:

- Xuất hiện hiện tượng "bẩn trạng thái" (State Pollution).
- Các dấu đánh dấu của nhánh trước vẫn còn tồn tại khi chuyển sang nhánh mới.
- Những vị trí hợp lệ có thể bị đánh dấu nhầm là không hợp lệ.
- Thuật toán bỏ sót nhiều nghiệm.
- Có thể kết luận sai rằng bài toán không có nghiệm.

- Số nghiệm tìm được có thể nhỏ hơn kết quả đúng.

Minh họa với bài toán n quân hậu:
Sau khi đặt quân hậu tại hàng i, cột j, chương trình đánh dấu:
col[j] = true
d1[i - j + n] = true
d2[i + j] = true
Sau khi gọi đệ quy:
Try(i + 1)
và nhánh đó kết thúc, phải hoàn tác ba trạng thái trên:
col[j] = false
d1[i - j + n] = false
d2[i + j] = false
Nếu bỏ qua ba dòng hoàn tác này, các dấu đánh dấu của quân hậu ở nhánh trước vẫn còn tồn tại.
Ví dụ, ở hàng 1 thử đặt quân hậu tại cột 1. Khi đó cột 1 và hai đường chéo liên quan được đánh dấu là đang bị chiếm.
Nếu nhánh này không tạo ra nghiệm và thuật toán quay lui để thử đặt quân hậu ở cột 2, các dấu đánh dấu của cột 1 vẫn còn nếu không thực hiện thao tác Undo.
Khi đó chương trình sẽ hiểu sai rằng những vị trí liên quan vẫn đang bị quân hậu chiếm hoặc kiểm soát. Các nhánh tiếp theo bị cắt tỉa sai, dẫn đến bỏ sót nghiệm.
KẾT LUẬN
Một thuật toán quay lui kinh điển cần phối hợp đầy đủ ba thành phần:

- Điều kiện dừng: xác định khi nào nghiệm đã hoàn chỉnh và kết thúc một nhánh.
- Điều kiện chấp nhận được: kiểm tra ràng buộc và loại bỏ các nhánh không hợp lệ càng sớm càng tốt.
- Khôi phục trạng thái: hoàn tác các thay đổi sau khi quay lui để các nhánh tiếp theo được xét trên trạng thái chính xác.

Nếu thiếu một trong ba thành phần, thuật toán có thể không kết thúc, sinh nghiệm sai, chạy chậm hoặc bỏ sót nghiệm.

Câu 1.3 (1,0 điểm) — Quay lui so với nhánh và cận

a) Phân biệt tiêu chí cắt nhánh của quay lui và của nhánh và cận. Vì sao quay lui dùng được cho bài toán liệt kê, còn nhánh và cận thì không?

b) Phát biểu điều kiện để một hàm cận là hợp lệ. Điều gì xảy ra nếu hàm tính cận dưới cho giá trị lớn hơn chi phí thật của phương án tối ưu trong nhánh? Lấy một ví dụ cụ thể trên ma trận chi phí ở mục D.

a) Phân biệt tiêu chí cắt nhánh và phạm vi áp dụng

Tiêu chí cắt nhánh của Quay lui (Backtracking):

Quay lui dựa chủ yếu vào tính khả thi của nghiệm bộ phận đối với các ràng buộc của bài toán.

Một nhánh tìm kiếm bị cắt ngay khi trạng thái hiện tại vi phạm ít nhất một ràng buộc của bài toán hoặc không thể tiếp tục tạo thành một nghiệm hợp lệ.

Ví dụ, trong bài toán n quân hậu, nếu quân hậu mới đặt trùng cột hoặc trùng đường chéo với một quân hậu đã đặt thì trạng thái đó không hợp lệ và nhánh tương ứng được cắt bỏ.

Tiêu chí cắt nhánh của Nhánh và Cận (Branch and Bound):

Nhánh và Cận dựa trên khả năng đạt được nghiệm tốt hơn nghiệm tốt nhất hiện tại (Best).

Trong bài toán tìm cực tiểu, nếu cận dưới của một nhánh không nhỏ hơn chi phí tốt nhất hiện tại thì nhánh đó có thể bị cắt:

Lower_Bound >= Best

Điều này có nghĩa là ngay cả trong trường hợp tốt nhất, nhánh đang xét cũng không thể tạo ra phương án có chi phí nhỏ hơn Best.

Trong bài toán tìm cực đại, điều kiện tương ứng là:

Upper_Bound <= Best

Khi đó nhánh không thể tạo ra nghiệm tốt hơn Best nên có thể loại bỏ.

Sự khác nhau:

- Quay lui: cắt nhánh chủ yếu dựa trên tính hợp lệ của nghiệm bộ phận.
- Nhánh và Cận: cắt nhánh dựa trên khả năng cải thiện nghiệm tốt nhất hiện tại thông qua hàm cận.
- Quay lui thường được sử dụng cho các bài toán tìm một lời giải, liệt kê tất cả lời giải hoặc đếm số lời giải.
- Nhánh và Cận chủ yếu được sử dụng cho các bài toán tối ưu.

Vì sao quay lui dùng được cho bài toán liệt kê, còn nhánh và cận thì không?

Bài toán liệt kê yêu cầu tìm và xuất ra toàn bộ các nghiệm hợp lệ. Vì vậy, mọi nhánh vẫn có khả năng chứa nghiệm hợp lệ đều phải được xét.

Quay lui chỉ cắt các nhánh vi phạm ràng buộc. Do đó các nghiệm hợp lệ không bị loại bỏ chỉ vì chúng có giá trị không tốt bằng một nghiệm khác.

Trong khi đó, Nhánh và Cận sử dụng Best để loại bỏ những nhánh không thể tạo ra nghiệm tốt hơn nghiệm tốt nhất hiện tại.

Nếu áp dụng cơ chế cắt theo Best vào bài toán liệt kê, một nhánh vẫn có thể chứa nghiệm hợp lệ nhưng sẽ bị loại bỏ chỉ vì nó không thể cải thiện Best. Khi đó các nghiệm hợp lệ trong nhánh đó sẽ bị bỏ sót.

Vì vậy, Nhánh và Cận với cơ chế cắt nhánh dựa trên hàm cận không phù hợp để liệt kê toàn bộ nghiệm. Ngược lại, Quay lui có thể dùng cho bài toán liệt kê vì nó chỉ loại bỏ những nhánh không thể tạo ra nghiệm hợp lệ.

b) Điều kiện của hàm cận, hệ quả khi hàm cận vi phạm và ví dụ minh họa

Điều kiện để một hàm cận là hợp lệ:

Đối với bài toán tìm cực tiểu như bài toán Người du lịch (TSP), hàm cận dưới phải là một cận dưới hợp lệ (admissible hoặc optimistic).

Gọi g(u) là cận dưới tại nút u và f*(u) là chi phí nhỏ nhất thực sự có thể đạt được của một phương án hoàn chỉnh phát triển từ nút u.

Hàm cận hợp lệ phải thỏa mãn:

g(u) <= f*(u)

Nói cách khác, cận dưới không được phép lớn hơn chi phí thực tế nhỏ nhất của bất kỳ phương án hoàn chỉnh nào phát triển từ nút đó.

Hàm cận phải có tính chất "lạc quan": nó có thể đánh giá thấp chi phí thực tế nhưng không được đánh giá cao hơn chi phí tối ưu thực tế của nhánh.

Điều gì xảy ra nếu hàm tính cận dưới cho giá trị lớn hơn chi phí thật?

Nếu:

g(u) > f*(u)

thì hàm cận đã đánh giá quá cao chi phí của nhánh.

Khi đó có thể xảy ra trường hợp:

Best <= g(u)

trong khi thực tế:

f*(u) < Best

Thuật toán sẽ cho rằng nhánh u không thể tạo ra nghiệm tốt hơn Best và cắt nhánh.

Tuy nhiên, thực tế nhánh đó lại chứa một phương án có chi phí nhỏ hơn Best.

Kết quả là thuật toán cắt nhầm nhánh chứa nghiệm tối ưu, dẫn đến việc không tìm được nghiệm tối ưu toàn cục và trả về một nghiệm có chất lượng kém hơn nghiệm tối ưu thực sự.

Ví dụ cụ thể trên ma trận chi phí mục D

Ma trận chi phí:

```
0   1   2   3   4
0 0 10 9 22 23
1 10 0 16 19 14
2 9 16 0 7 28
3 22 19 7 0 8
4 23 14 28 8 0
```

Xét nhánh:

0 -> 2 -> 3

Chi phí đã đi:
c(0,2) + c(2,3) = 9 + 7 = 16
Các thành phố chưa đi qua là 1 và 4.
Một phương án hoàn chỉnh tốt từ nhánh này là:
0 -> 2 -> 3 -> 4 -> 1 -> 0
Chi phí của phần còn lại:
c(3,4) + c(4,1) + c(1,0)
= 8 + 14 + 10
= 32
Do đó chi phí thực tế của toàn bộ chu trình là:
f* = 16 + 32 = 48
Giả sử trước đó thuật toán đã tìm được một chu trình có:
Best = 55
Bây giờ giả sử hàm cận bị sai và đánh giá chi phí phần còn lại là:
42
Trong khi chi phí phần còn lại thực tế chỉ là:
32
Khi đó hàm cận tính:
g = 16 + 42 = 58
Thuật toán kiểm tra:
g >= Best
58 >= 55
Điều kiện đúng nên thuật toán cắt nhánh:
0 -> 2 -> 3
Tuy nhiên, đây là một quyết định sai vì nhánh này thực tế chứa chu trình:
0 -> 2 -> 3 -> 4 -> 1 -> 0
với chi phí:
9 + 7 + 8 + 14 + 10 = 48
Mà:
48 < 55
Như vậy nhánh bị cắt thực tế chứa một nghiệm tốt hơn Best hiện tại.
Hậu quả:
Thuật toán không thể tìm được nghiệm có chi phí 48 và có thể kết thúc với nghiệm có chi phí 55.
Điều này chứng minh rằng nếu hàm cận dưới đánh giá quá cao chi phí thực tế thì Nhánh và Cận có thể cắt nhầm nhánh chứa nghiệm tối ưu, từ đó làm mất tính đúng đắn của thuật toán tối ưu.
Kết luận:

Đối với bài toán cực tiểu, hàm cận dưới phải thỏa mãn:
g(u) <= f*(u)
Nếu hàm cận lớn hơn chi phí tối ưu thực tế của một nhánh thì có nguy cơ cắt nhầm nhánh chứa nghiệm tối ưu.
Do đó, tính hợp lệ của hàm cận là điều kiện quan trọng để thuật toán Nhánh và Cận vẫn đảm bảo tìm được nghiệm tối ưu.

Phần 2. Bài toán n quân hậu theo bảy bước chuẩn (4,0 điểm)

Trình bày chi tiết thuật toán quay lui giải bài toán n quân hậu theo đầy đủ bảy bước học thuật của môn. Bàn cờ dùng để lần vết bằng tay là n = 4; phần chạy chương trình làm với n = 5, 6 và 8.

Bước 1 và 2 — Phát biểu bài toán hình thức và ý tưởng giải thuật

Bước 1 phải nêu đủ: dữ liệu vào, dữ liệu ra, điều kiện để một cách đặt là hợp lệ, và cách biểu diễn lời giải. Bước 2 phải trả lời được: vì sao biểu diễn lời giải bằng một **hoán vị** lại thu gọn được không gian tìm kiếm, và thu gọn từ bao nhiêu xuống bao nhiêu với n = 8.

BƯỚC 1 — PHÁT BIỂU BÀI TOÁN HÌNH THỨC

- Dữ liệu vào: Số nguyên dương n (n >= 4), kích thước bàn cờ n x n.
- Dữ liệu ra: Tập hợp tất cả các bộ tọa độ đặt n quân hậu sao cho không quân nào khống chế quân nào.
- Điều kiện hợp lệ: Không có 2 quân hậu nào cùng hàng, cùng cột hoặc cùng đường chéo:
  - Khác cột: ci != cj (với mọi i != j).
  - Khác đường chéo: |i - j| != |ci - cj| (với mọi i != j).
- Biểu diễn lời giải: Vectơ X = (x1,x2,...,xn), trong đó xi thuộc {1,2,...,n} là vị trí cột của quân hậu tại hàng i. Quân hậu thứ i đặt tại ô (i,xi).

BƯỚC 2 — Ý TƯỞNG GIẢI THUẬT VÀ THU GỌN KHÔNG GIAN TÌM KIẾM

- Ý tưởng giải thuật:
  - Dùng quay lui: lần lượt đặt từng quân hậu từ hàng 1 đến hàng n.
  - Tại hàng i, thử đặt vào cột j thuộc {1,...,n}.
  - Nếu ô (i,j) không bị khống chế bởi các quân hậu ở các hàng trước, thông qua kiểm tra cột, đường chéo xuôi i-j và đường chéo ngược i+j, ta gán xi = j, đánh dấu trạng thái và gọi đệ quy sang hàng i+1.
  - Sau khi duyệt xong nhánh, bỏ đánh dấu (undo) để thử cột khác.
  - Khi i > n, ta đã có một nghiệm hoàn chỉnh và ghi nhận nghiệm.
- Vì sao biểu diễn bằng hoán vị thu gọn được không gian tìm kiếm:
  - Ràng buộc bài toán đòi hỏi không có 2 quân hậu nào cùng cột (xi != xk với mọi i != k).
  - Do đó, tập {x1,...,xn} bắt buộc phải là một hoán vị của {1,...,n}.
  - Việc ép nghiệm thành dạng hoán vị loại bỏ ngay các trường hợp trùng cột, không cần duyệt qua các trạng thái này.
- Mức độ thu gọn với n = 8:
  - Không gian ban đầu:
    S1 = 8^8 = 16.777.216 trạng thái.
  - Không gian sau khi ép hoán vị:
    S2 = 8! = 40.320 trạng thái.
  - Hệ số thu gọn:
    S1 / S2 = 16.777.216 / 40.320 ≈ 416,1 lần.
  - Tỷ lệ giảm:
    (1 - S2/S1) x 100% ≈ 99,76%.

=> Như vậy, biểu diễn nghiệm bằng hoán vị giúp thu gọn không gian tìm kiếm khoảng 416 lần, loại bỏ khoảng 99,76% các trạng thái trùng cột.

Bước 3 và 4 — Giả mã chuẩn và chứng minh tính đúng đắn

Bước 3: viết giả mã theo đúng quy ước của bài giảng (chỉ số đánh từ 1), kèm bảng giải thích vai trò của từng dòng lệnh, **chỉ rõ dòng nào là phép cắt và dòng nào là thao tác hoàn tác**.
Bước 4: chứng minh thuật toán liệt kê đủ và không lặp — mỗi lời giải hợp lệ được sinh ra đúng một lần.

Bước 3 — giả mã N-QUAN-HAU

```
Thuật toán N-Queens(n)
Dữ liệu: Số nguyên dương n
Dữ liệu ra: Liệt kê tất cả các cấu hình X = (x[1], x[2], ..., x[n]) hợp lệ

1.  Khởi tạo mảng toàn cục:
2.      x[1..n]
3.      col[1..n] ← false
4.      diag1[-(n-1) .. (n-1)] ← false    // Hoặc diag1[1 .. 2n-1] với độ
lệch +n
5.      diag2[2 .. 2n] ← false
6.
7.  Thủ tục Try(i):
8.      Nếu i > n thì:
9.          Ghi nhận hoặc in nghiệm (x[1], x[2], ..., x[n])
10.         Trả về
11.     Với j chạy từ 1 đến n làm:
12.         Nếu (không col[j]) và (không diag1[i - j]) và (không diag2[i +
j]) thì:
13.             x[i] ← j
14.             col[j] ← true
15.             diag1[i - j] ← true
16.             diag2[i + j] ← true
17.             Try(i + 1)
18.             col[j] ← false
19.             diag1[i - j] ← false
20.             diag2[i + j] ← false
21.
22. Bắt đầu:
23.     Try(1)
```

Dòng	Lệnh	Vai trò của dòng lệnh
1-5	`Khai báo & khởi tạo x, col, diag1, diag2`	Khởi tạo cấu trúc dữ liệu toàn cục để kiểm tra xung đột trong thời gian $O(1)$.
7	`Thủ tục Try(i)`	Bắt đầu thủ tục quay lui đặt quân hậu tại hàng thứ i.
8-10	`Nếu i > n thì: Ghi nhận nghiệm; Trả về`	**Điều kiện dừng (Neo đệ quy):** Đã xếp thành công cả n quân hậu từ hàng 1 đến n.
11	`Với j chạy từ 1 đến n làm:`	Xét lần lượt tất cả các khả năng đặt quân hậu vào cột j tại hàng i.
12	`Nếu (không col[j]) và (không diag1[i - j]) và ...`	**Phép cắt tỉa (Pruning):** Kiểm tra an toàn; nếu vi phạm cùng cột hoặc cùng đường chéo thì bỏ qua ngay lập tức, cắt toàn bộ cây con phía dưới.
13-16	`x[i] ← j; col[j] ← true; diag1[...] ← true; ...`	**Ghi nhận trạng thái:** Đặt quân hậu vào ô (i, j) và khóa cột j, hai đường chéo đi qua ô đó.
17	`Try(i + 1)`	**Bước tiến đệ quy:** Chuyển sang tìm vị trí đặt quân hậu tiếp theo ở hàng $i + 1$.
18-20	`Bắt đầu: Try(1)`	Điểm kích hoạt chương trình, bắt đầu xếp quân hậu từ hàng đầu tiên.

Bảng sinh viên tự điền: mỗi dòng giả mã một hàng.

Bước 4 — chứng minh liệt kê đủ và không lặp

BƯỚC 4 — CHỨNG MINH TÍNH ĐÚNG ĐẮN (LIỆT KÊ ĐỦ VÀ KHÔNG LẶP)

- Liệt kê đủ (Không bỏ sót nghiệm):

Giả sử X* = (x*1, ..., x*n) là một nghiệm hợp lệ tùy ý. Vì X* không có xung đột giữa bất kỳ cặp quân hậu nào, nên tại mỗi bước i (1 <= i <= n), việc gán x[i] = x*i luôn thỏa mãn điều kiện an toàn ở dòng 12 và không bị cắt nhánh. Do đó, thuật toán duyệt tuần tự qua mọi tiền tố của X* và chắc chắn đi tới lá i = n + 1 để ghi nhận nghiệm. Vì X* là nghiệm hợp lệ bất kỳ nên mọi nghiệm hợp lệ đều được tìm thấy.

- Không trùng lặp (Mỗi nghiệm sinh đúng một lần):

Mỗi nghiệm hoàn chỉnh tương ứng với một đường đi duy nhất từ gốc xuống lá trên cây không gian trạng thái. Tại mỗi nút cha ở mức i, vòng lặp xét các giá trị cột j thuộc {1,...,n}; mỗi giá trị chỉ được thử đúng một lần tại nút đó. Các nhánh có giá trị j khác nhau là các nhánh phân biệt, nên hai đường đi khác nhau sẽ tạo ra hai cấu hình khác nhau. Thuật toán duyệt theo DFS nên mỗi đường đi tới lá chỉ được duyệt và ghi nhận đúng một lần.

KẾT LUẬN:

Mọi nghiệm hợp lệ đều được sinh ra và mỗi nghiệm chỉ được sinh đúng một lần. Vì vậy, thuật toán N-Queens liệt kê đầy đủ và không lặp các nghiệm.

Bước 5 — Bảng lần vết trên bàn cờ n = 4

Lần vết bằng tay, không chạy chương trình. Mỗi hàng của bảng là **một lần xét một ô**: ghi rõ đang ở hàng nào, thử cột nào, ô ấy có bị quân đã đặt ăn hay không, và việc làm tương ứng (đặt quân xuống, bỏ qua vì bị ăn, hay gỡ quân ra khi quay lui).

Phân biệt hai con số dễ lẫn

Số ô đã xét là số hàng của bảng lần vết: mỗi lần thử một cột tính một ô.

Số nút của cây quay lui là số lần **gọi hàm đệ quy**. Một lời gọi hàm ứng với một hàng của bàn cờ và bên trong nó thử tối đa n cột, nên hai con số này khác nhau. Bảng tổng kết bên dưới hỏi cả hai; ghi nhầm chỗ là mất điểm.

Bước	Hàng i	Cột c	Bị ăn?	Việc làm	Trạng thái x = (x_1, x_2, x_3, x_4)
1	1	1	không	Đặt quân X1=1; gọi Try(2)	(1, -, -, -)
2	2	3	không	Ô 1, 2 bị ăn; Đặt quân X2=3; gọi Try(3)	(1, 3, -, -)
3	3	-	có	Hàng 3 cả 4 cột đều bị ăn; Quay lui về hàng 2 (gỡ X 2=3)	(1, -, -, -)
4	2	4	không	Đặt quân X2=4; gọi Try(3)	(1, 4, -, -)

Bước	Hàng i	Cột c	Bị ăn?	Việc làm	Trạng thái x = (x_1, x_2, x_3, x_4)
5	3	2	không	Ô 1 bị ăn; Đặt quân X3=2; gọi Try(4)	(1, 4, 2, -)
6	4	-	có	Hàng 4 cả 4 cột đều bị ăn; Quay lui về hàng 3 (gỡ X3=2)	(1, 4, -, -)
7	2	-	có	Ô 3, 4 hàng 3 bị ăn; Quay lui về hàng 1 (gỡ X1=1)	(-, -, -, -)
8	1	2	không	Đặt quân X1=2; gọi Try(2)	(2, -, -, -)
9	2	4	không	Ô 1, 2, 3 bị ăn; Đặt quân X2=4; gọi Try(3)	(2, 4, -, -)
10	3	1	không	Đặt quân X3=1; gọi Try(4)	(2, 4, 1, -)
11	4	3	không	Ô 1, 2 bị ăn; **Ghi nhận nghiệm 1**: (2, 4, 1, 3)	(2, 4, 1, 3)
12	4	-	có	Quay lui: gỡ X4=2, gỡ X3=4, gỡ X2=1, gỡ X1=3	(-, -, -, -)
13	2	3	không	Đặt quân X1=3; gọi Try(2)	(3, -, -, -)
14	3	1	không	Đặt quân X2=1; gọi Try(3)	(3, 1, -, -)
15	4	4	không	Ô 1, 2, 3 bị ăn; Đặt quân X3=4; gọi Try(4)	(3, 1, 4, -)
16	5	2	không	Ô 1 bị ăn; **Ghi nhận nghiệm 2**: (3, 1, 4, 2)	(3, 1, 4, 2)
17	4	-	có	Quay lui: gỡ X4=2, gỡ X3=4, gỡ X2=1, gỡ X1=3	(-, -, -, -)
18	2	4	không	Thử X1=4; các nhánh con đều bị ăn; gỡ x_1, hoàn tất thuật toán	(-, -, -, -)

Bảng 2.1. Lần vết quay lui trên bàn cờ 4 × 4. Thêm hàng nếu thiếu chỗ.

Đại lượng	Giá trị sinh viên tìm được
Số ô đã xét (số hàng của bảng lần vết)	**18 bước trạng thái** (hoặc ghi chú: *tương ứng xét 60 lượt ô thử trên bàn cờ*)
Số nút của cây quay lui, tức số lần gọi hàm đệ quy	**17** lần gọi hàm *(gốc Try(1): 1; Try(2): 4; Try(3): 6; Try(4): 4; chạm đích Try(5): 2)*
Số ô bị loại vì bị ăn	**44** ô

Đại lượng	Giá trị sinh viên tìm được
Số lời giải và danh sách lời giải	**2 lời giải**: 1. (2, 4, 1, 3) 2. (3, 1, 4, 2)
So với vét cạn mọi hoán vị của 4 phần tử (4! = 24)	Nhánh cắt sớm ở ngay hàng 2 và 3, chỉ có **4 nhánh** đi tới được hàng 4 (sinh 2 nghiệm, 2 nhánh bế tắc), giảm bớt phần lớn số hoán vị phải kiểm tra so với vét cạn 24 cấu hình.

Bước 6 — Mã nguồn ba ngôn ngữ

Chương trình phải nhận n từ bàn phím, in ra **số nút đã duyệt**, **số lời giải** và bàn cờ của lời giải đầu tiên. Đếm nút theo đúng quy ước: mỗi lần gọi hàm đệ quy tính một nút.

Python — n_quan_hau.py

```
import sys

def main():
    try:
        line = sys.stdin.readline().strip()
        if not line:
            return
        n = int(line)
        if n <= 0:
            return
    except Exception:
        return

    full_mask = (1 << n) - 1
    x = [0] * n
    first_solution = [0] * n

    # Stack trạng thái cho mỗi hàng (lặp thay vì đệ quy để tránh overhead)
    cols_arr = [0] * (n + 1)
    d1_arr   = [0] * (n + 1)
    d2_arr   = [0] * (n + 1)
    avail_arr = [0] * (n + 1)
    avail_arr[0] = full_mask

    row = 0
    node_count = 1          # tương đương lời gọi backtrack(0)
    solution_count = 0

    while row >= 0:
        avail = avail_arr[row]
        if avail:
            bit = avail & -avail              # lấy bit thấp nhất
            avail_arr[row] = avail ^ bit      # xoá bit vừa lấy
            x[row] = bit.bit_length() - 1     # vị trí cột (0-indexed)

            new_cols = cols_arr[row] | bit
            new_d1   = ((d1_arr[row] | bit) << 1) & full_mask
            new_d2   = (d2_arr[row] | bit) >> 1

            if row == n - 1:                  # đặt xong hậu cuối -> lá
                solution_count += 1
```

```
            node_count += 1                # nút lá backtrack(n)
            if solution_count == 1:
                first_solution[:] = x
        else:                              # đi sâu hơn
            row += 1
            cols_arr[row] = new_cols
            d1_arr[row]   = new_d1
            d2_arr[row]   = new_d2
            avail_arr[row] = full_mask & ~(new_cols | new_d1 | new_d2)
            node_count += 1
    else:
        row -= 1                           # backtrack

  out = [f"So nut da duyet: {node_count}",
         f"So loi giai: {solution_count}"]
  if solution_count > 0:
      out.append("Ban co loi giai dau tien:")
      for i in range(n):
          out.append(" ".join("Q" if first_solution[i] == j else "."
                              for j in range(n)))
  else:
      out.append("Khong co loi giai nao.")
  sys.stdout.write("\n".join(out) + "\n")

if __name__ == "__main__":
  main()
"Ghi chú: Mã nguồn được tối ưu hóa bằng kỹ thuật Bit Manipulation kết hợp khử đệ
quy bằng mảng trạng thái (Iterative Bitmask Backtracking) nhằm loại bỏ chi phí gọi
hàm của Python, đồng thời vẫn bảo toàn chính xác quy tắc đếm số nút cây quay lui."
```

C++ — n_quan_hau.cpp

```
#include <iostream>
#include <vector>
#include <cstdint>

using namespace std;

int n;
long long node_count = 0;
long long solution_count = 0;
uint32_t full_mask;
vector<int> x;
vector<int> first_solution;

void backtrack(int row, uint32_t cols, uint32_t d1, uint32_t d2) {
    ++node_count;
    if (row == n) {
        ++solution_count;
        if (solution_count == 1) first_solution = x;
        return;
    }
    uint32_t avail = full_mask & ~(cols | d1 | d2);
    while (avail) {
        uint32_t bit = avail & -avail;        // bit thấp nhất
        avail ^= bit;
        x[row] = __builtin_ctz(bit);          // vị trí bit = số bit 0 ở cuối
        backtrack(row + 1,
                  cols | bit,
                  ((d1 | bit) << 1) & full_mask,
                  (d2 | bit) >> 1);
    }
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n) || n <= 0 || n > 31) return 0;

    x.assign(n, 0);
    first_solution.assign(n, 0);
    full_mask = (uint32_t(1) << n) - 1;
```

```
    backtrack(0, 0, 0, 0);

    cout << "So nut da duyet: " << node_count << "\n"
         << "So loi giai: " << solution_count << "\n";
    if (solution_count > 0) {
        cout << "Ban co loi giai dau tien:\n";
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j)
                cout << (first_solution[i] == j ? "Q " : ". ");
            cout << "\n";
        }
    } else {
        cout << "Khong co loi giai nao.\n";
    }
    return 0;
}
```

Julia — n_quan_hau.jl (chú ý chỉ số đánh từ 1)

```
function main()
    line = readline()
    if isempty(strip(line))
        return
    end
    n = parse(Int, strip(line))
    if n <= 0 || n > 31
        return
    end

    full_mask = (UInt32(1) << n) - UInt32(1)
    x = zeros(Int, n)
    first_solution = zeros(Int, n)
    node_count     = Ref{Int}(0)
    solution_count = Ref{Int}(0)

    function backtrack(row::Int, cols::UInt32, d1::UInt32, d2::UInt32)
        node_count[] += 1
        if row > n
            solution_count[] += 1
            if solution_count[] == 1
                first_solution .= x
            end
            return
        end
        avail = full_mask & ~(cols | d1 | d2)
        while avail != 0
            bit = avail & -avail
            avail ⊻= bit
            x[row] = trailing_zeros(bit) + 1        # 1-indexed
            backtrack(row + 1,
                      cols | bit,
                      ((d1 | bit) << 1) & full_mask,
                      (d2 | bit) >> 1)
        end
    end

    backtrack(1, UInt32(0), UInt32(0), UInt32(0))

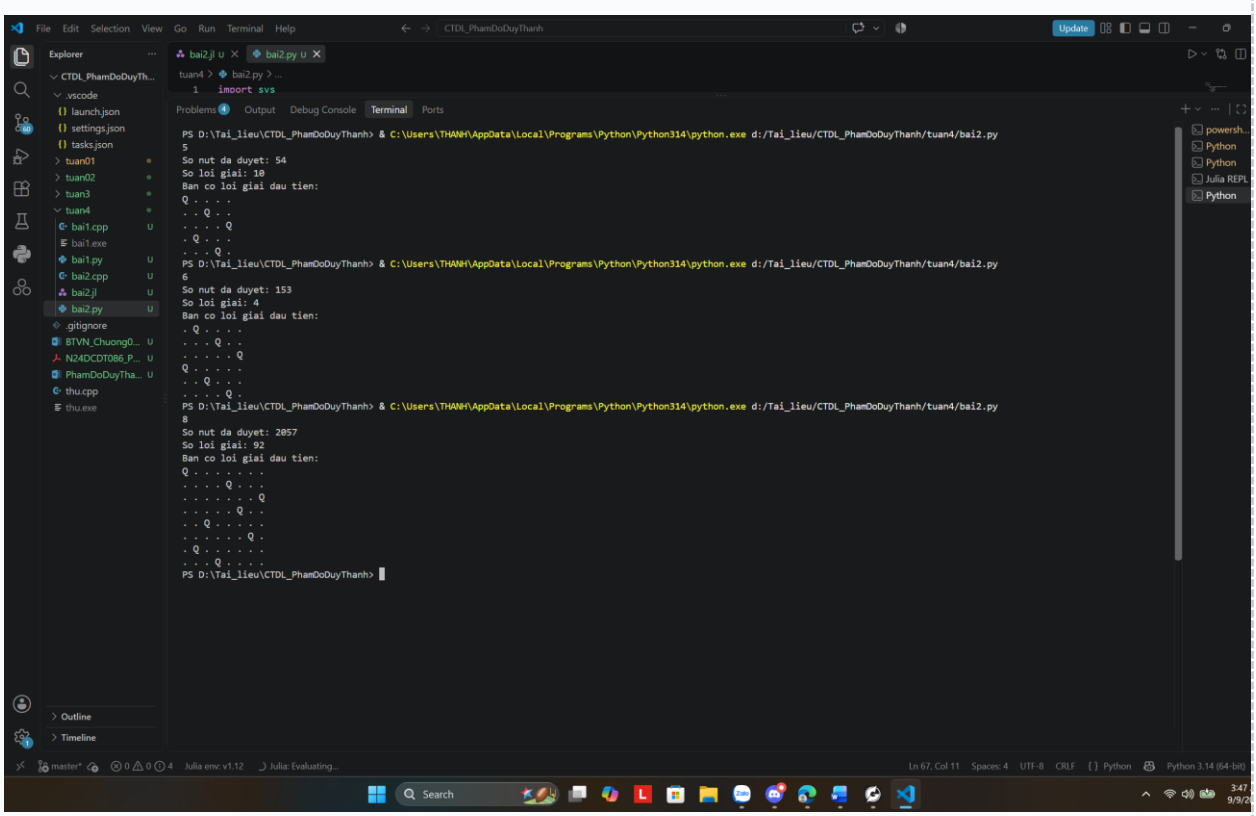
    println("So nut da duyet: ", node_count[])
    println("So loi giai: ", solution_count[])
    if solution_count[] > 0
        println("Ban co loi giai dau tien:")
        for i in 1:n
            row = [first_solution[i] == j ? "Q" : "." for j in 1:n]
            println(join(row, " "))
        end
    else
        println("Khong co loi giai nao.")
    end
end

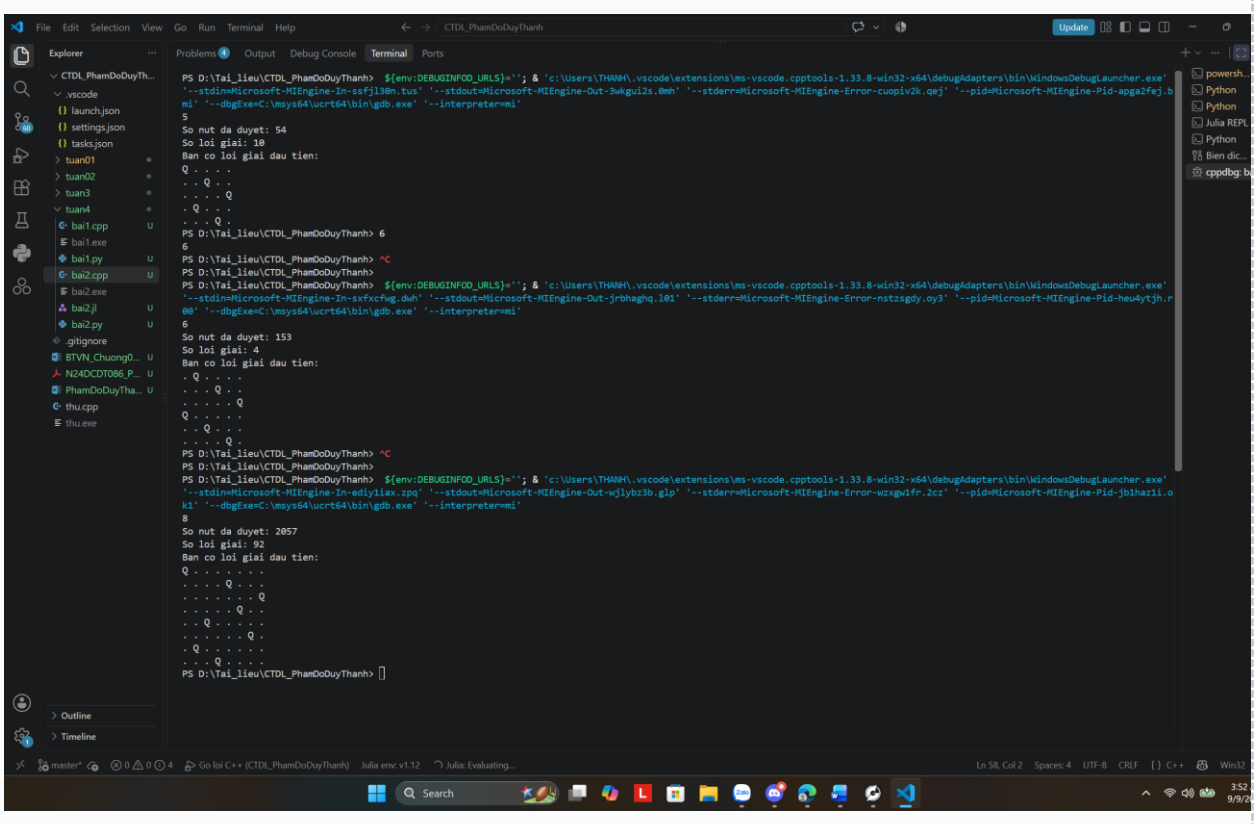
main()
```

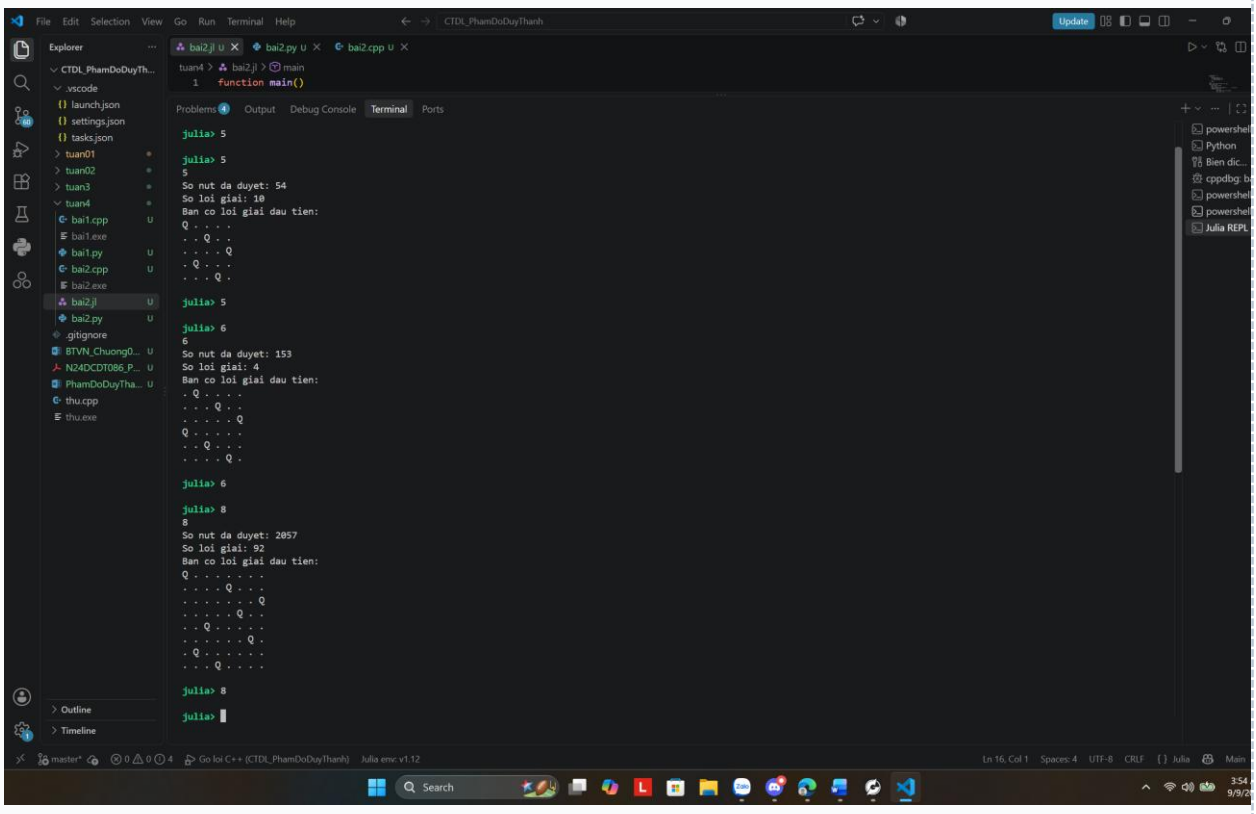
Bước 7 — Kết quả chạy thực nghiệm

Chạy chương trình với n = 5, 6 và 8, điền số liệu vào bảng rồi dán ảnh chụp màn hình.

n	Số nút đã duyệt	Số lời giải	Số hoán vị nếu vét cạn (n!)	Tỉ lệ nút / n!
5	54	10	120	0,5250
6	153	4	720	0,2125
8	2057	92	40.320	0,0510


PS D:\Tai_lieu\CTDL_PhamDoDuyThanh> & C:\Users\THANH\AppData\Local\Programs\Python\Python314\python.exe d:/Tai_lieu/CTDL_PhamDoDuyThanh/tuan4/bai2.py
5
So nut da duyet: 54
So loi giai: 10
Ban co loi giai dau tien:
Q . . . .
. . Q . .
. . . . Q
. Q . . .
. . . Q .
PS D:\Tai_lieu\CTDL_PhamDoDuyThanh> & C:\Users\THANH\AppData\Local\Programs\Python\Python314\python.exe d:/Tai_lieu/CTDL_PhamDoDuyThanh/tuan4/bai2.py
6
So nut da duyet: 153
So loi giai: 4
Ban co loi giai dau tien:
. Q . . . .
. . . Q . .
. . . . . Q
Q . . . . .
. . Q . . .
. . . . Q .
PS D:\Tai_lieu\CTDL_PhamDoDuyThanh> & C:\Users\THANH\AppData\Local\Programs\Python\Python314\python.exe d:/Tai_lieu/CTDL_PhamDoDuyThanh/tuan4/bai2.py
8
So nut da duyet: 2057
So loi giai: 92
Ban co loi giai dau tien:
Q . . . . . .
. . . . Q . . .
. . . . . Q
. . . . . Q .
. . Q . . . .
. . . . . Q .
. Q . . . . .
. . . Q . . .
PS D:\Tai_lieu\CTDL_PhamDoDuyThanh>


PS D:\Tai_lieu\CTDL_PhamDoDuyThanh> 6
6
PS D:\Tai_lieu\CTDL_PhamDoDuyThanh> ^C
PS D:\Tai_lieu\CTDL_PhamDoDuyThanh>
PS D:\Tai_lieu\CTDL_PhamDoDuyThanh> 6
So nut da duyet: 153
So loi giai: 4
Ban co loi giai dau tien:
. Q . . . .
. . . Q . .
. . . . . Q
Q . . . . .
. . Q . . .
. . . . Q .
PS D:\Tai_lieu\CTDL_PhamDoDuyThanh> ^C
PS D:\Tai_lieu\CTDL_PhamDoDuyThanh>
PS D:\Tai_lieu\CTDL_PhamDoDuyThanh>
8
So nut da duyet: 2057
So loi giai: 92
Ban co loi giai dau tien:
Q . . . . . .
. . . . Q . . .
. . . . . Q
. . . . . Q .
. . Q . . . .
. . . . . Q .
. Q . . . . .
. . . Q . . .
PS D:\Tai_lieu\CTDL_PhamDoDuyThanh>



Nhận xét: nhìn cột cuối, khi n tăng thì phép cắt nhánh giúp được nhiều hơn hay ít đi? Giải thích vì sao.

Khi n tăng, phép cắt nhánh **giúp được nhiều hơn vượt trội** (hiệu quả cắt tỉa tăng mạnh).

Giải thích:

- **Tỉ lệ nút duyệt giảm mạnh:** Khi n tăng từ 5 lên 8, tỉ lệ Số nút/n! giảm từ **52,50%** (n = 5) xuống **21,25%** (n = 6) và chỉ còn **5,10%** (n = 8). Trong khi không gian vét cạn tăng theo hàm giai thừa O(n!), số nút thực tế được duyệt tăng chậm hơn đáng kể. Vì vậy, tỉ lệ này có xu hướng giảm về 0 khi n tăng.
- **Ràng buộc ngày càng dày đặc:** Mỗi quân hậu khi được đặt sẽ chiếm một cột và hai đường chéo. Khi kích thước bàn cờ tăng, số vị trí bị khống chế cũng tăng, làm số ô an toàn ở các hàng tiếp theo giảm nhanh.
- **Hiệu quả của cắt tỉa sớm (Early Pruning):** Khi phát hiện xung đột tại hàng thứ k, thuật toán loại bỏ ngay toàn bộ nhánh con phía dưới thay vì tiếp tục xét từng trạng thái. Nhánh bị loại có thể chứa tới (n - k)! trạng thái nếu vét cạn. Do đó, cắt tỉa càng sớm thì càng tiết kiệm đáng kể số phép thử và thời gian tính toán.

Phần 3. Lần vết ba thuật toán sinh kế tiếp (1,5 điểm)

Cả ba câu làm **bằng tay**, không chạy chương trình. Với mỗi bước sinh phải ghi rõ vị trí i tìm được và phần đuôi bị sửa lại.

Câu 3.1 (0,5 điểm) — Sinh xâu nhị phân kế tiếp

Cho xâu nhị phân 10110. Sinh ba xâu kế tiếp theo thứ tự từ điển. Mỗi bước ghi rõ vị trí chữ số 0 đầu tiên tính từ phải sang, và những chữ số 1 nào bị xoá về 0. Giải thích vì sao quy tắc này tương đương với phép cộng thêm 1 trong hệ nhị phân.

Bước	Xâu hiện tại	Vị trí i	Các chữ số 1 bị xoá về 0	Xâu kế tiếp
1	10110	5	Không có (sau vị trí 5 không còn bit nào)	10111
2	10111	2	Bit tại vị trí 3, 4, 5 (các bit 1 ở đuôi chuyển thành 0)	11000
3	11000	5	Không có (sau vị trí 5 không còn bit nào)	11001

Giải thích quan hệ với phép cộng thêm 1

Sinh xâu nhị phân kế tiếp chính là mô phỏng phép cộng thêm 1 trong hệ nhị phân:

- Nếu bit cuối cùng là 0: đổi 0 thành 1 và kết thúc.
- Nếu bit cuối cùng là 1: đổi 1 thành 0 và truyền số nhớ sang trái.
- Số nhớ tiếp tục lan qua các bit 1 liên tiếp cho đến khi gặp bit 0. Bit 0 đó được đổi thành 1.

Vì vậy, thuật toán lùi từ phải sang trái để tìm bit 0 đầu tiên tại vị trí i, đổi xi thành 1 và đưa toàn bộ các bit phía sau về 0 chính là cách mô phỏng cơ chế truyền nhớ của phép cộng thêm 1.

Câu 3.2 (0,5 điểm) — Sinh tổ hợp kế tiếp

Cho tổ hợp (1, 3, 6, 8) với n = 8, k = 4. Sinh ba tổ hợp kế tiếp. Mỗi bước ghi rõ vị trí i đầu tiên chưa chạm trần tính từ phải sang, giá trị trần n − k + i của vị trí ấy, và phần đuôi được tính lại thế nào.

Bước	Tổ hợp hiện tại	Vị trí i	Trần n − k + i	Tổ hợp kế tiếp
1	(1, 3, 6, 8)	3	37	(1, 3, 7, 8)
2	(1, 3, 7, 8)	2	6	(1, 4, 5, 6)
3	(1, 4, 5, 6)	4	8	(1, 4, 5, 7)

Phần đuôi sau vị trí i được tính lại theo quy tắc nào, và vì sao quy tắc ấy cho tổ hợp nhỏ nhất còn lại?

Quy tắc tính lại phần đuôi:

Sau khi tăng ai lên 1, các phần tử phía sau được gán:

aj = aj-1 + 1, với j = i + 1, ..., k.

Giải thích:

Các phần tử của tổ hợp luôn thỏa mãn:

a1 < a2 < ... < ak.

Do đó, sau khi tăng ai, để tạo ra tổ hợp nhỏ nhất nhưng vẫn lớn hơn tổ hợp hiện tại, mỗi phần tử phía sau phải nhận giá trị nhỏ nhất có thể. Vì vậy:

ai+1 = ai + 1

ai+2 = ai+1 + 1

...

Cách này tạo ra phần đuôi nhỏ nhất có thể và đảm bảo tổ hợp thu được là tổ hợp kế tiếp theo thứ tự từ điển.

Câu 3.3 (0,5 điểm) — Sinh hoán vị kế tiếp

Cho hoán vị (3, 1, 5, 4, 2). Sinh ba hoán vị kế tiếp. Mỗi bước ghi rõ chỉ số i, chỉ số j, hai phần tử bị đổi chỗ và đoạn đuôi bị đảo ngược. Sau đó trả lời: bỏ hẳn bước đảo ngược phần đuôi thì chuyện gì xảy ra?

Bước	Hoán vị hiện tại	i	j	Sau khi đổi chỗ	Hoán vị kế tiếp
1	(3, 1, 5, 4, 2)	2	5	Đổi a1=2 với a5=2: **(3, 2, 5, 4, 1)**	Đảo đoạn 3…5 (đoạn 5, 4, 1): **(3, 2, 1, 4, 5)**
2	(3, 2, 1, 4, 5)	4	5	Đổi a4=4 với a5=5: **(3, 2, 1, 5, 4)**	Đảo đoạn 5…5 (đoạn 4): **(3, 2, 1, 5, 4)**
3	(3, 2, 1, 5, 4)	3	5	Đổi a3=1 với a5=4: **(3, 2, 4, 5, 1)**	Đảo đoạn 4…5 (đoạn 5, 1): **(3, 2, 4, 1, 5)**

Bỏ bước đảo ngược phần đuôi thì chuyện gì xảy ra?

Nếu bỏ bước đảo ngược phần đuôi từ vị trí i + 1 đến n:

1. **Hoán vị sinh ra không phải là cấu hình kế tiếp nhỏ nhất:**
   Sau khi đổi chỗ a[i] và a[j], đoạn đuôi từ i + 1 đến n vẫn ở trạng thái giảm dần. Đây là thứ tự lớn nhất của các phần tử trong đoạn đuôi, trong khi để tạo hoán vị kế tiếp theo thứ tự từ điển, đoạn đuôi phải được sắp xếp tăng dần.
2. **Bỏ sót nhiều hoán vị hợp lệ:**
   Nếu giữ nguyên đoạn đuôi giảm dần, thuật toán sẽ nhảy qua các hoán vị trung gian nhỏ hơn. Vì vậy, không thể duyệt lần lượt toàn bộ n! hoán vị theo đúng thứ tự từ điển.
3. **Phá vỡ tính đúng đắn của thuật toán sinh kế tiếp:**
   Thuật toán vẫn có thể tiếp tục sinh ra các hoán vị, nhưng các hoán vị không còn là hoán vị kế tiếp theo thứ tự từ điển. Do đó, bước đảo ngược đoạn đuôi là bắt buộc để bảo đảm mỗi lần sinh đều cho ra cấu hình kế tiếp nhỏ nhất.

Phần 4. Ba bài toán ứng dụng trên ba ngôn ngữ (1,5 điểm)

Cả ba bài đều phải có đủ ba bản Python, C++ và Julia. Mỗi chương trình in ra **số nút đã duyệt** bên cạnh đáp số. Con số ấy quan trọng ngang đáp số, vì nó cho thấy phép cắt có thật sự hoạt động hay không.

Bài 4.1 (0,5 điểm) — Bài toán cái túi bằng nhánh và cận

Cài đặt bài toán cái túi bằng nhánh và cận trên bộ đồ vật ở mục C với W = 14. Dùng hàm cận trên kiểu tham lam phân số. Chương trình phải chạy hai chế độ trên cùng dữ liệu: có cắt nhánh và không cắt nhánh, rồi so số nút của hai chế độ. Trả lời thêm: vì sao bắt buộc phải sắp xếp đồ vật theo tỉ số v/w giảm dần trước khi duyệt?

Đại lượng	Kết quả
Thứ tự xét đồ vật sau khi sắp theo v/w giảm dần	A → B → E → D → C → F
Giá trị lớn nhất và tập đồ vật được chọn	Giá trị = 53; Tập đồ vật = {A, B, E}; Tổng trọng lượng = 14
Số nút khi CÓ cắt nhánh	7 nút
Số nút khi KHÔNG cắt nhánh	89 nút
Số nút giảm được bao nhiêu lần	89 / 7 ≈ 12,71 lần =~ 92,13%

Python

```
class Item:
    def __init__(self, name, w, v):
        self.name = name
        self.w = w
        self.v = v
        self.ratio = v / w

# Bộ dữ liệu chuẩn mục C của đề bài
W = 14
raw_data = [
    ('A', 4, 19),
    ('B', 7, 25),
    ('C', 3, 7),
    ('D', 3, 8),
    ('E', 3, 9),
    ('F', 10, 21)
]

items = [Item(name, w, v) for name, w, v in raw_data]
items.sort(key=lambda x: x.ratio, reverse=True)
n = len(items)

max_val = 0
best_mask = 0
nodes_visited = 0

def bound(level, cw, cv):
    if cw > W:
        return 0
    b = cv
    w = cw
    for i in range(level, n):
        if w + items[i].w <= W:
            w += items[i].w
            b += items[i].v
        else:
            b += (W - w) * items[i].ratio
            break
    return b

def dfs(level, cw, cv, mask, prune):
```

```
    global max_val, best_mask, nodes_visited
    nodes_visited += 1

    if cv > max_val:
        max_val = cv
        best_mask = mask
    if level == n:
        return

    if prune and bound(level, cw, cv) <= max_val:
        return

    # Nhánh 1: Chọn đồ vật level (thử trước vì tỉ số v/w cao)
    if cw + items[level].w <= W:
        dfs(level + 1, cw + items[level].w, cv + items[level].v, mask | (1 << level), prune)

    # Nhánh 2: Không chọn đồ vật level
    dfs(level + 1, cw, cv, mask, prune)

def solve(prune):
    global max_val, best_mask, nodes_visited
    max_val = 0
    best_mask = 0
    nodes_visited = 0
    dfs(0, 0, 0, 0, prune)

    print(f"Giá trị lớn nhất: {max_val}")
    selected = [items[i].name for i in range(n) if (best_mask & (1 << i))]
    print(f"Tập đồ vật được chọn: {selected}")
    print(f"Số nút đã duyệt: {nodes_visited}")

print("--- KHÔNG CẮT NHÁNH ---")
solve(False)

print("\n--- CÓ CẮT NHÁNH ---")
solve(True)
```

C++

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

struct Item {
    char name;
    double w, v;
    double ratio;
};

double W = 14;
vector<Item> items;
int n;

double max_val;
int best_mask;
int nodes_visited;

double bound(int level, double cw, double cv) {
    if (cw > W) return 0;
    double b = cv;
    double w = cw;
    for (int i = level; i < n; i++) {
        if (w + items[i].w <= W) {
            w += items[i].w;
            b += items[i].v;
        } else {
            b += (W - w) * items[i].ratio;
            break;
        }
    }
    return b;
}

void dfs(int level, double cw, double cv, int mask, bool prune) {
    nodes_visited++;
    if (cv > max_val) {
        max_val = cv;
        best_mask = mask;
    }
    if (level == n) return;

    if (prune && bound(level, cw, cv) <= max_val) {
        return;
    }

    if (cw + items[level].w <= W) {
        dfs(level + 1, cw + items[level].w, cv + items[level].v, mask | (1 <<
level), prune);
```

```
    }
    dfs(level + 1, cw, cv, mask, prune);
}

void solve(bool prune) {
    max_val = 0;
    best_mask = 0;
    nodes_visited = 0;
    dfs(0, 0, 0, 0, prune);

    cout << "Gia tri lon nhat: " << max_val << endl;
    cout << "Tap do vat chon: ";
    for (int i = 0; i < n; i++) {
        if (best_mask & (1 << i)) {
            cout << items[i].name << " ";
        }
    }
    cout << endl;
    cout << "So nut da duyet: " << nodes_visited << endl;
}

int main() {
    // Dữ liệu chuẩn mục C của đề bài
    items = {
        {'A', 4, 19, 19.0 / 4},
        {'B', 7, 25, 25.0 / 7},
        {'C', 3, 7,  7.0 / 3},
        {'D', 3, 8,  8.0 / 3},
        {'E', 3, 9,  9.0 / 3},
        {'F', 10, 21, 21.0 / 10}
    };
    n = items.size();

    // Sắp xếp giảm dần theo tỉ số v/w
    sort(items.begin(), items.end(), [](const Item& a, const Item& b){
        return a.ratio > b.ratio;
    });

    cout << "--- KHONG CAT NHANH ---" << endl;
    solve(false);

    cout << "\n--- CO CAT NHANH ---" << endl;
    solve(true);

    return 0;
}
```

Julia (chỉ số đánh từ 1)

```
struct Item
    name::String
    w::Float64
    v::Float64
    ratio::Float64
end

# Dữ liệu chuẩn mục C của đề bài
const W = 14.0
raw_items = [
    Item("A", 4.0, 19.0, 19.0 / 4.0),
    Item("B", 7.0, 25.0, 25.0 / 7.0),
    Item("C", 3.0,  7.0,  7.0 / 3.0),
    Item("D", 3.0,  8.0,  8.0 / 3.0),
    Item("E", 3.0,  9.0,  9.0 / 3.0),
    Item("F", 10.0, 21.0, 21.0 / 10.0)
]

items = sort(raw_items, by = x -> x.ratio, rev = true)
n = length(items)

max_val = 0.0
best_mask = 0
nodes_visited = 0

function bound(level::Int, cw::Float64, cv::Float64)
    if cw > W return 0.0 end
    b = cv
    w_curr = cw
    for i in level:n
        if w_curr + items[i].w <= W
            w_curr += items[i].w
            b += items[i].v
        else
            b += (W - w_curr) * items[i].ratio
            break
        end
    end
    return b
end
```

```
function dfs(level::Int, cw::Float64, cv::Float64, mask::Int, prune::Bool)
    global max_val, best_mask, nodes_visited
    nodes_visited += 1

    if cv > max_val
        max_val = cv
        best_mask = mask
    end
    if level > n return end

    if prune && bound(level, cw, cv) <= max_val
        return
    end

    if cw + items[level].w <= W
        dfs(level + 1, cw + items[level].w, cv + items[level].v, mask | (1 << (level - 1)),
prune)
    end
    dfs(level + 1, cw, cv, mask, prune)
end

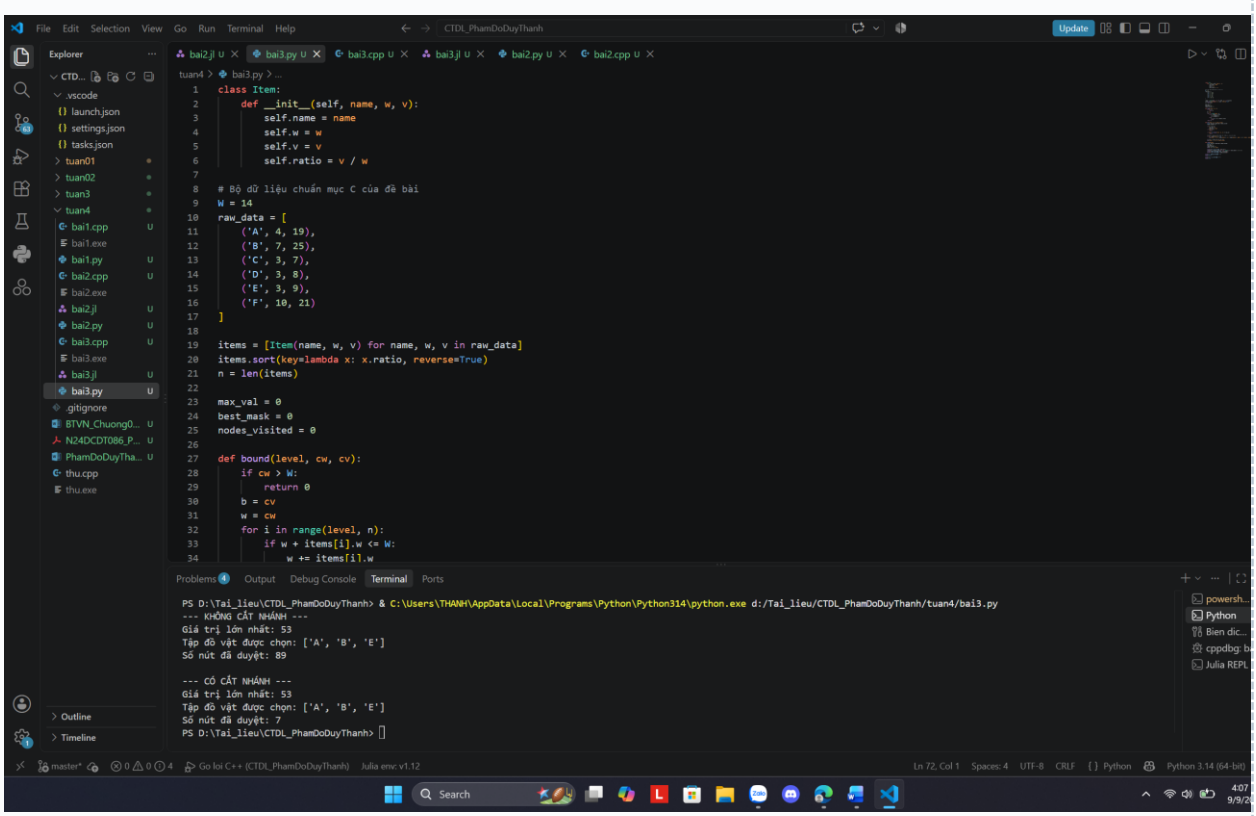
function solve(prune::Bool)
    global max_val = 0.0
    global best_mask = 0
    global nodes_visited = 0
    dfs(1, 0.0, 0.0, 0, prune)

    println("Gia tri lon nhat: ", max_val)
    print("Tap do vat chon: ")
    for i in 1:n
        if best_mask & (1 << (i - 1)) != 0
            print(items[i].name, " ")
        end
    end
    println()
    println("So nut da duyet: ", nodes_visited)
end

println("--- KHONG CAT NHANH ---")
solve(false)

println("\n--- CO CAT NHANH ---")
```

```
solve(true)
```



```python
class Item:
    def __init__(self, name, w, v):
        self.name = name
        self.w = w
        self.v = v
        self.ratio = v / w

# Bộ dữ liệu chuẩn mục C của đề bài
W = 14
raw_data = [
    ('A', 4, 19),
    ('B', 7, 25),
    ('C', 3, 7),
    ('D', 3, 8),
    ('E', 3, 9),
    ('F', 10, 21)
]

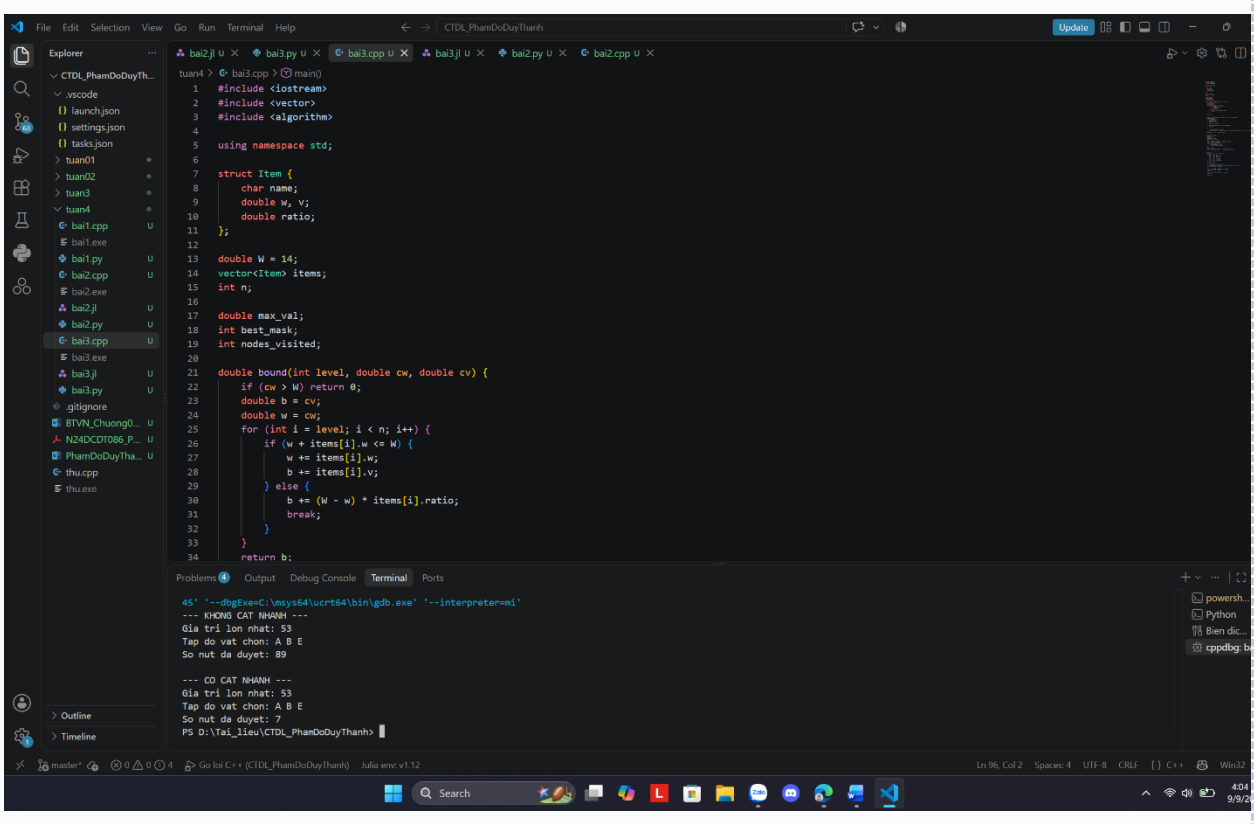
items = [Item(name, w, v) for name, w, v in raw_data]
items.sort(key=lambda x: x.ratio, reverse=True)
n = len(items)

max_val = 0
best_mask = 0
nodes_visited = 0

def bound(level, cw, cv):
    if cw > W:
        return 0
    b = cv
    w = cw
    for i in range(level, n):
        if w + items[i].w <= W:
            w += itemsfi1.w
```

```
PS D:\Tai_lieu\CTDL_PhamDoDuyThanh> & C:\Users\THANH\AppData\Local\Programs\Python\Python314\python.exe d:/Tai_lieu/CTDL_PhamDoDuyThanh/tuan4/bai3.py
--- KHÔNG CẮT NHÁNH ---
Giá trị lớn nhất: 53
Tập đồ vật được chọn: ['A', 'B', 'E']
Số nút đã duyệt: 89

--- CÓ CẮT NHÁNH ---
Giá trị lớn nhất: 53
Tập đồ vật được chọn: ['A', 'B', 'E']
Số nút đã duyệt: 7
PS D:\Tai_lieu\CTDL_PhamDoDuyThanh>
```



```cpp
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

struct Item {
    char name;
    double w, v;
    double ratio;
};

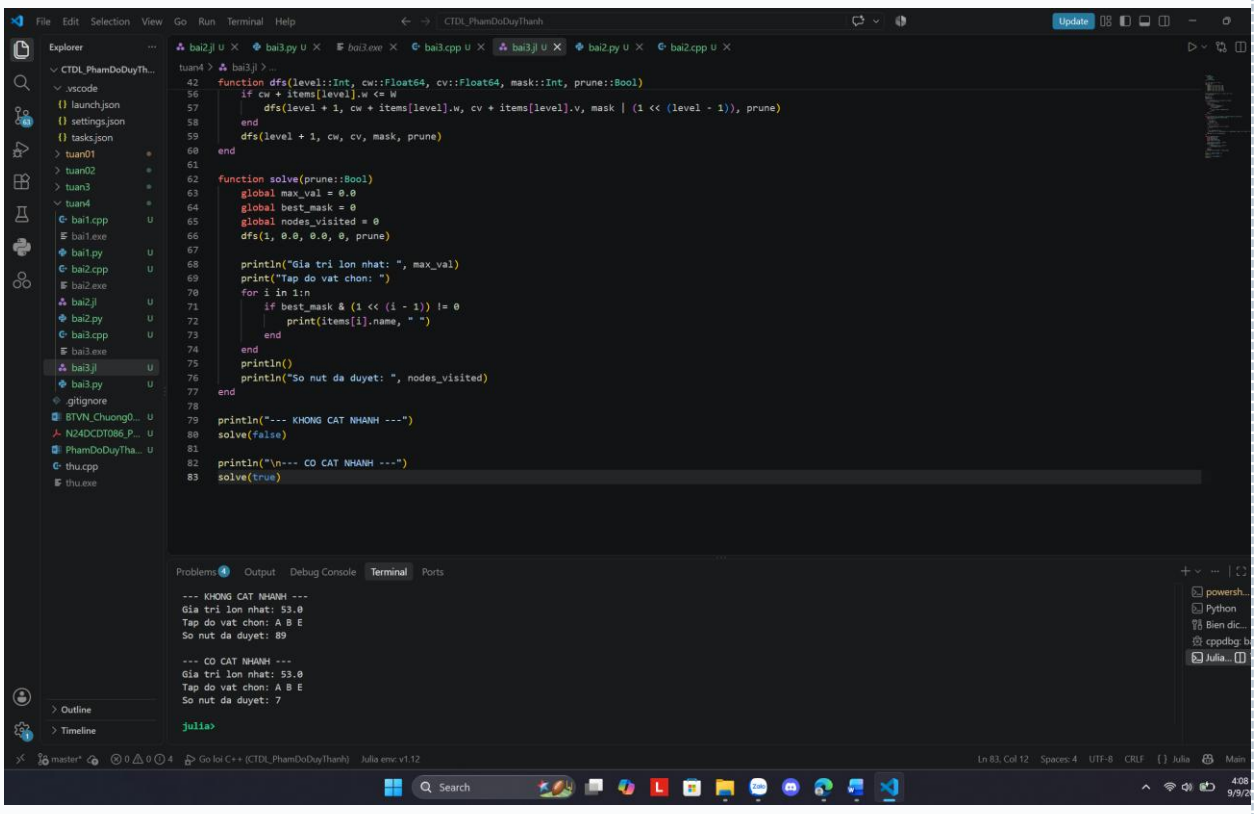
double W = 14;
vector<Item> items;
int n;

double max_val;
int best_mask;
int nodes_visited;

double bound(int level, double cw, double cv) {
    if (cw > W) return 0;
    double b = cv;
    double w = cw;
    for (int i = level; i < n; i++) {
        if (w + items[i].w <= W) {
            w += items[i].w;
            b += items[i].v;
        } else {
            b += (W - w) * items[i].ratio;
            break;
        }
    }
    return b;
```

```
45' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
--- KHONG CAT NHANH ---
Gia tri lon nhat: 53
Tap do vat chon: A B E
So nut da duyet: 89

--- CO CAT NHANH ---
Gia tri lon nhat: 53
Tap do vat chon: A B E
So nut da duyet: 7
PS D:\Tai_lieu\CTDL_PhamDoDuyThanh>
```



Vì sao phải sắp xếp theo v/w giảm dần?
Phải sắp xếp đồ vật theo v/w giảm dần vì hàm cận trên sử dụng **tham lam phân số** để ước lượng giá trị lớn nhất mà một nhánh còn có thể đạt được.
Khi sắp xếp theo v/w giảm dần, thuật toán ưu tiên các đồ vật có **giá trị trên mỗi đơn vị trọng lượng cao nhất** trước. Phần trọng lượng còn lại được lấp đầy theo thứ tự này, cho phép tính được một **cận trên hợp lệ** cho nhánh đang xét.
Nếu cận trên không chính xác hoặc thấp hơn giá trị tốt nhất mà nhánh có thể đạt được, thuật toán có thể **cắt nhầm nhánh chứa nghiệm tối ưu**, dẫn đến sai đáp án.
Ngoài ra, việc sắp xếp theo v/w giảm dần giúp cận trên được tính nhanh và thường đạt giá trị chặt hơn, từ đó **tăng hiệu quả cắt nhánh**.

Bài 4.2 (0,5 điểm) — Bài toán người du lịch bằng nhánh và cận

Cài đặt bài toán người du lịch trên ma trận chi phí ở mục D, xuất phát và kết thúc ở thành phố 0. Dùng hàm cận dưới đơn giản nhất: chi phí đã đi cộng với số cạnh còn phải đi nhân cạnh rẻ nhất toàn ma trận. Chạy hai chế độ có cắt và không cắt, so số nút. Trả lời thêm: chứng minh hàm cận này hợp lệ, tức nó không bao giờ lớn hơn chi phí thật của phương án tốt nhất trong nhánh.

Đại lượng	Kết quả
Cạnh rẻ nhất toàn ma trận c_min	7 (cạnh giữa thành phố 2 và 3: c(2,3) = c(3,2) =7)

Đại lượng	Kết quả
Chi phí hành trình tối ưu	48
Hành trình tối ưu	0 → 2 → 3 → 4 → 1 → 0
Số nút khi CÓ cắt nhánh	42 nút
Số nút khi KHÔNG cắt nhánh	65 nút

Python

```
import math

# Ma trận chi phí chuẩn mục D của đề bài
matrix = [
    [0, 10, 9, 22, 23],
    [10, 0, 16, 19, 14],
    [9, 16, 0, 7, 28],
    [22, 19, 7, 0, 8],
    [23, 14, 28, 8, 0]
]
N = len(matrix)

# Tìm c_min (cạnh rẻ nhất toàn ma trận, bỏ qua đường chéo chính)
c_min = math.inf
for i in range(N):
    for j in range(N):
        if i != j and matrix[i][j] < c_min:
            c_min = matrix[i][j]

best_cost = math.inf
best_path = []
nodes_visited = 0

def dfs(city, visited, cost, path, prune):
    global best_cost, best_path, nodes_visited
    nodes_visited += 1

    # Số cạnh còn phải đi để hoàn thành chu trình quay về 0
    edges_left = N - (len(path) - 1)
    lower_bound = cost + edges_left * c_min

    # Cắt nhánh nếu cận dưới >= kỷ lục hiện tại
    if prune and lower_bound >= best_cost:
        return

    # Nếu đã đi qua đủ N thành phố, đóng chu trình quay về 0
    if len(path) == N:
        final_cost = cost + matrix[city][0]
        if final_cost < best_cost:
            best_cost = final_cost
            best_path = path[:] + [0]
        return

    # Thử các thành phố tiếp theo
    for next_city in range(N):
        if not (visited & (1 << next_city)):
            dfs(next_city, visited | (1 << next_city),
                cost + matrix[city][next_city], path + [next_city], prune)

def solve(prune):
    global best_cost, best_path, nodes_visited
```

```
    best_cost = math.inf
    best_path = []
    nodes_visited = 0
    dfs(0, 1 << 0, 0, [0], prune)

    print(f"Chi phí tối ưu: {best_cost}")
    print(f"Hành trình: {' -> '.join(map(str, best_path))}")
    print(f"Số nút đã duyệt: {nodes_visited}")

print(f"Canh re nhat toan ma tran c_min: {c_min}\n")

print("--- KHÔNG CẮT NHÁNH ---")
solve(False)

print("\n--- CÓ CẮT NHÁNH ---")
solve(True)
```

C++

```
#include <iostream>
#include <vector>
#include <climits>

using namespace std;

// Ma trận chi phí chuẩn mục D của đề bài
vector<vector<int>> matrix = {
    {0, 10, 9, 22, 23},
    {10, 0, 16, 19, 14},
    {9, 16, 0, 7, 28},
    {22, 19, 7, 0, 8},
    {23, 14, 28, 8, 0}
};
int N = matrix.size();
int c_min = INT_MAX;

int best_cost;
vector<int> best_path;
int nodes_visited;

void dfs(int city, int visited, int cost, vector<int>& path, bool prune) {
    nodes_visited++;

    int edges_left = N - (path.size() - 1);
    int lower_bound = cost + edges_left * c_min;

    if (prune && lower_bound >= best_cost) {
        return;
    }

    if (path.size() == N) {
        int final_cost = cost + matrix[city][0];
        if (final_cost < best_cost) {
            best_cost = final_cost;
            best_path = path;
            best_path.push_back(0);
        }
        return;
    }
```

```
    for (int next_city = 0; next_city < N; next_city++) {
        if (!(visited & (1 << next_city))) {
            path.push_back(next_city);
            dfs(next_city, visited | (1 << next_city), cost + matrix[city][next_city], path,
prune);
            path.pop_back();
        }
    }
}

void solve(bool prune) {
    best_cost = INT_MAX;
    best_path.clear();
    nodes_visited = 0;
    vector<int> path = {0};
    dfs(0, 1 << 0, 0, path, prune);

    cout << "Chi phi toi uu: " << best_cost << endl;
    cout << "Hanh trinh: ";
    for (size_t i = 0; i < best_path.size(); i++) {
        cout << best_path[i] << (i == best_path.size() - 1 ? "" : " -> ");
    }
    cout << endl;
    cout << "So nut da duyet: " << nodes_visited << endl;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (i != j && matrix[i][j] < c_min) {
                c_min = matrix[i][j];
            }
        }
    }

    cout << "Canh re nhat toan ma tran c_min: " << c_min << "\n\n";

    cout << "--- KHONG CAT NHANH ---" << endl;
    solve(false);
```

```
    cout << "\n--- CO CAT NHANH ---" << endl;
    solve(true);

    return 0;
}
```

Julia (chỉ số đánh từ 1)

```
# Ma trận chuẩn mục D của đề bài (Julia đánh chỉ số từ 1)
matrix = [
    0 10  9 22 23;
   10  0 16 19 14;
    9 16  0  7 28;
   22 19  7  0  8;
   23 14 28  8  0
]
N = size(matrix, 1)

# Tìm c_min (bỏ qua đường chéo chính)
c_min = minimum(matrix[i, j] for i in 1:N for j in 1:N if i != j)

best_cost = typemax(Int)
best_path = Int[]
nodes_visited = 0

function dfs(city::Int, visited::Int, cost::Int, path::Vector{Int}, prune::Bool)
    global best_cost, best_path, nodes_visited
    nodes_visited += 1

    # Số cạnh còn lại cần đi: N - (độ dài path - 1)
    edges_left = N - (length(path) - 1)
    lower_bound = cost + edges_left * c_min

    # Điều kiện cắt nhánh
    if prune && lower_bound >= best_cost
        return
    end

    # Nếu đã đi qua đủ N thành phố, quay về thành phố 1 (tương ứng đỉnh 0)
    if length(path) == N
        final_cost = cost + matrix[city, 1]
        if final_cost < best_cost
            best_cost = final_cost
            best_path = push!(copy(path), 1)
        end
        return
    end

    # Thử các thành phố tiếp theo
    for next_city in 1:N
        if (visited & (1 << (next_city - 1))) == 0
```

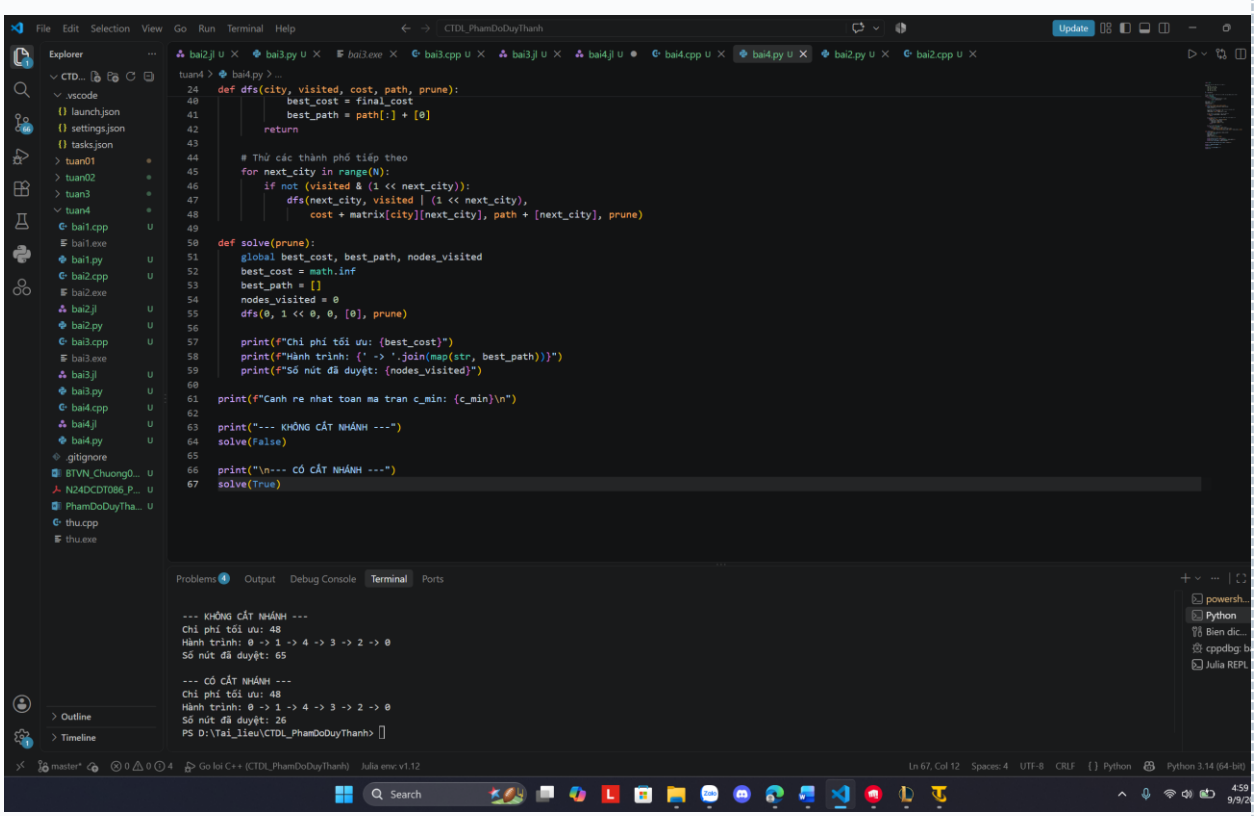
```
            dfs(next_city, visited | (1 << (next_city - 1)),
                cost + matrix[city, next_city], push!(copy(path), next_city), prune)
        end
    end
end

function solve(prune::Bool)
    global best_cost = typemax(Int)
    global best_path = Int[]
    global nodes_visited = 0
    dfs(1, 1 << 0, 0, [1], prune)

    println("Chi phí tối ưu: ", best_cost)
    print("Hành trình: ")
    # Trừ đi 1 để in ra dạng 0-based khớp với ma trận đề bài
    println(join([string(c - 1) for c in best_path], " -> "))
    println("Số nút đã duyệt: ", nodes_visited)
end

println("Cạnh rẻ nhất c_min: ", c_min)
println("\n--- KHÔNG CẮT NHÁNH ---")
solve(false)

println("\n--- CÓ CẮT NHÁNH ---")
solve(true)
```



```
def dfs(city, visited, cost, path, prune):
            best_cost = final_cost
            best_path = path[:] + [0]
        return

    # Thử các thành phố tiếp theo
    for next_city in range(N):
        if not (visited & (1 << next_city)):
            dfs(next_city, visited | (1 << next_city),
                cost + matrix[city][next_city], path + [next_city], prune)

def solve(prune):
    global best_cost, best_path, nodes_visited
    best_cost = math.inf
    best_path = []
    nodes_visited = 0
    dfs(0, 1 << 0, 0, [0], prune)

    print(f"Chi phí tối ưu: {best_cost}")
    print(f"Hành trình: {' -> '.join(map(str, best_path))}")
    print(f"Số nút đã duyệt: {nodes_visited}")

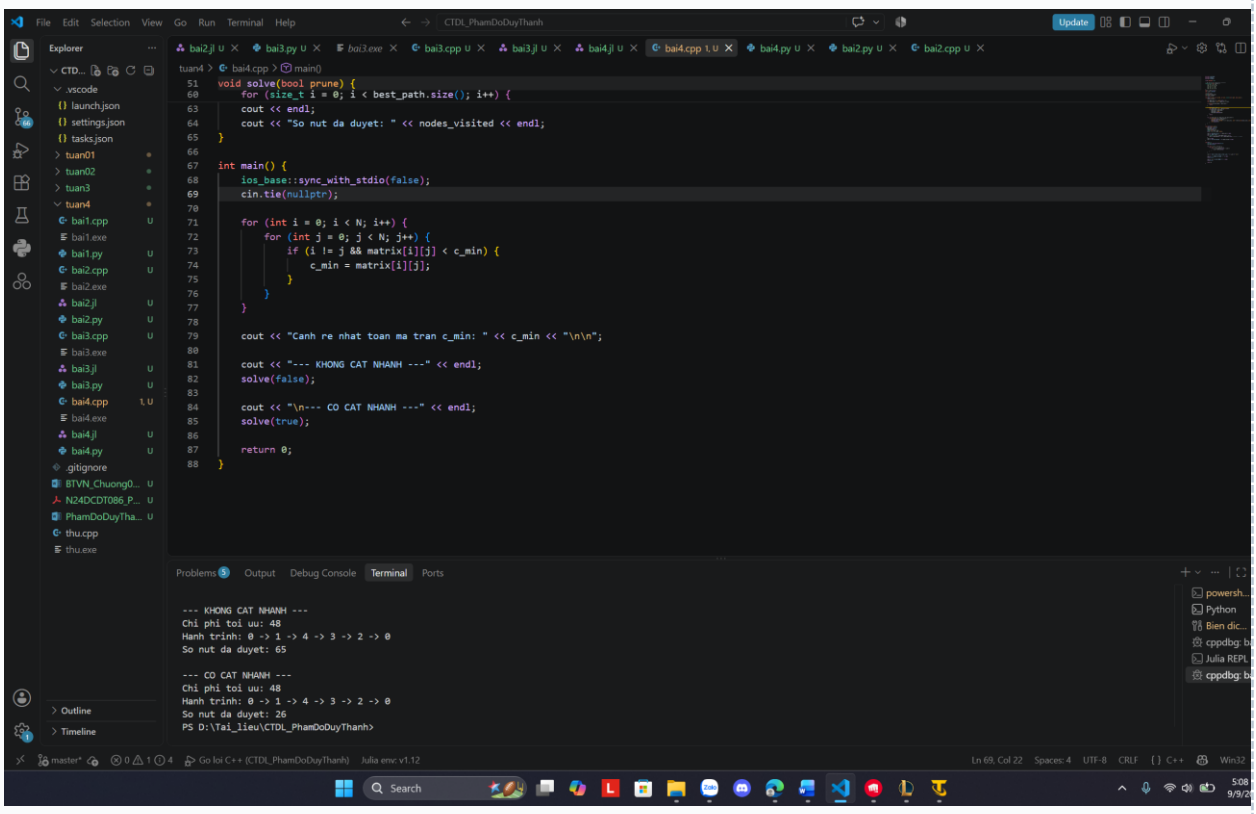
print(f"Canh re nhat toan ma tran c_min: {c_min}\n")

print("--- KHÔNG CẮT NHÁNH ---")
solve(False)

print("\n--- CÓ CẮT NHÁNH ---")
solve(True)
```

```
--- KHÔNG CẮT NHÁNH ---
Chi phí tối ưu: 48
Hành trình: 0 -> 1 -> 4 -> 3 -> 2 -> 0
Số nút đã duyệt: 65

--- CÓ CẮT NHÁNH ---
Chi phí tối ưu: 48
Hành trình: 0 -> 1 -> 4 -> 3 -> 2 -> 0
Số nút đã duyệt: 26
PS D:\Tai_lieu\CTDL_PhamDoDuyThanh>
```



```
void solve(bool prune) {
    for (size_t i = 0; i < best_path.size(); i++) {
    cout << endl;
    cout << "So nut da duyet: " << nodes_visited << endl;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (i != j && matrix[i][j] < c_min) {
                c_min = matrix[i][j];
            }
        }
    }

    cout << "Canh re nhat toan ma tran c_min: " << c_min << "\n\n";

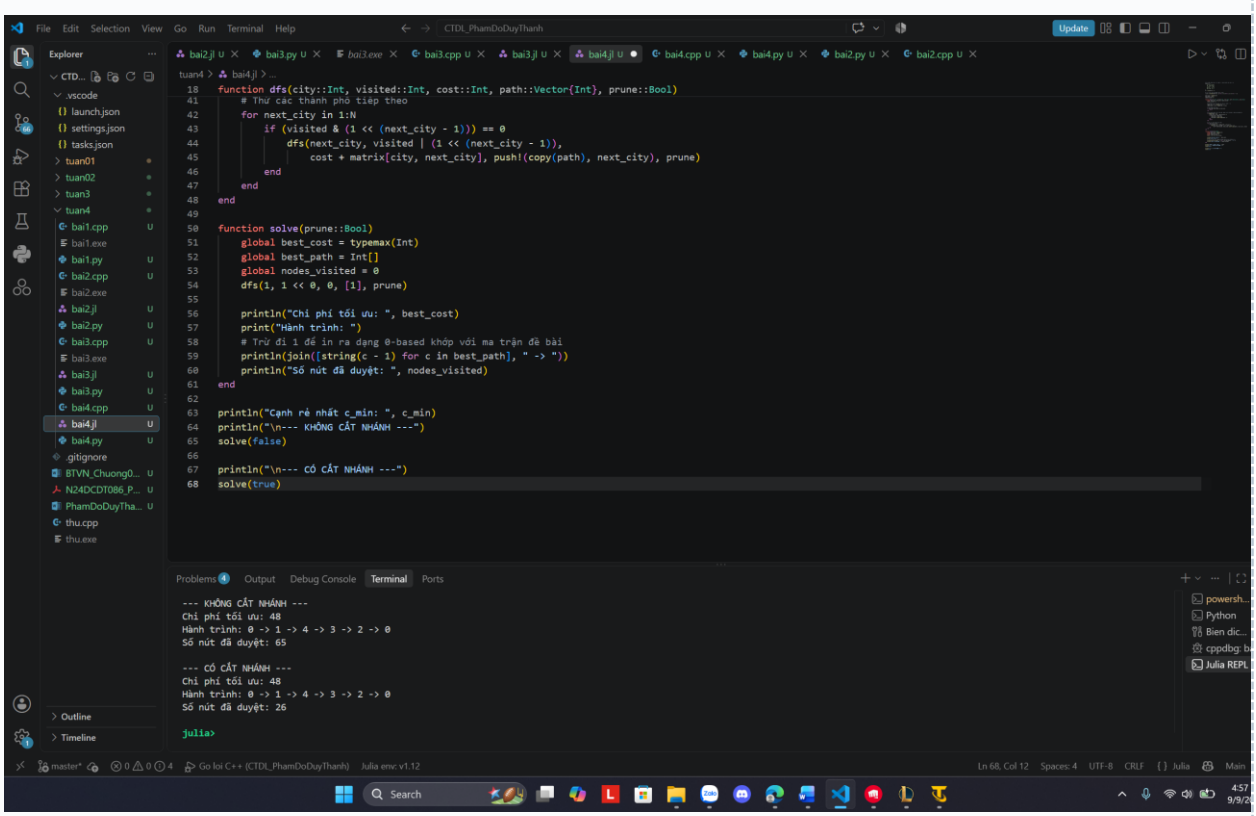
    cout << "--- KHONG CAT NHANH ---" << endl;
    solve(false);

    cout << "\n--- CO CAT NHANH ---" << endl;
    solve(true);

    return 0;
}
```

```
--- KHONG CAT NHANH ---
Chi phi toi uu: 48
Hanh trinh: 0 -> 1 -> 4 -> 3 -> 2 -> 0
So nut da duyet: 65

--- CO CAT NHANH ---
Chi phi toi uu: 48
Hanh trinh: 0 -> 1 -> 4 -> 3 -> 2 -> 0
So nut da duyet: 26
PS D:\Tai_lieu\CTDL_PhamDoDuyThanh>
```


function dfs(city::Int, visited::Int, cost::Int, path::Vector{Int}, prune::Bool)
Thử các thành phố tiếp theo
for next_city in 1:N
if (visited & (1 << (next_city - 1))) == 0
dfs(next_city, visited | (1 << (next_city - 1)),
cost + matrix[city, next_city], push!(copy(path), next_city), prune)
end
end
end
function solve(prune::Bool)
global best_cost = typemax(Int)
global best_path = Int[]
global nodes_visited = 0
dfs(1, 1 << 0, 0, [1], prune)
println("Chi phí tối ưu: ", best_cost)
print("Hành trình: ")
Trừ đi 1 để in ra dạng 0-based khớp với ma trận đề bài
println(join([string(c - 1) for c in best_path], " -> "))
println("Số nút đã duyệt: ", nodes_visited)
end
println("Cạnh rẻ nhất c_min: ", c_min)
println("\n--- KHÔNG CẮT NHÁNH ---")
solve(false)
println("\n--- CÓ CẮT NHÁNH ---")
solve(true)
--- KHÔNG CẮT NHÁNH ---
Chi phí tối ưu: 48
Hành trình: 0 -> 1 -> 4 -> 3 -> 2 -> 0
Số nút đã duyệt: 65
--- CÓ CẮT NHÁNH ---
Chi phí tối ưu: 48
Hành trình: 0 -> 1 -> 4 -> 3 -> 2 -> 0
Số nút đã duyệt: 26
julia>

Chứng minh hàm cận dưới hợp lệ (Admissible Lower Bound)
Xét một nút trên cây tìm kiếm, đã đi qua một lộ trình bộ phận với k thành phố và chi phí tích lũy là cost. Để hoàn thành chu trình và quay về thành phố 0, cần đi tiếp m = N - k + 1 cạnh, tương ứng với biến edges_left trong chương trình.
1. Chi phí thực tế của mọi chu trình hoàn chỉnh
Với mọi phương án hoàn chỉnh mở rộng từ nhánh hiện tại:
f = cost + c(e1) + c(e2) + ... + c(em)
trong đó e1, e2, ..., em là các cạnh còn lại cần đi.
2. So sánh với cận dưới
Vì c_min là trọng số cạnh nhỏ nhất trong toàn bộ ma trận nên:
c(e) >= c_min với mọi cạnh e.
Do đó:
c(e1) + c(e2) + ... + c(em) >= m × c_min
Suy ra:
f >= cost + m × c_min
Mà m = edges_left, nên:
f >= cost + edges_left × c_min = lower_bound
3. Kết luận
Ta có: lower_bound <= f
Với mọi phương án hoàn chỉnh có thể mở rộng từ nhánh hiện tại. Vì vậy, lower_bound không bao giờ lớn hơn chi phí thực tế của nghiệm tốt nhất trong nhánh, nên đây là một **cận dưới hợp lệ (admissible lower bound)**.
Do đó, khi lower_bound >= Best, thuật toán có thể cắt nhánh mà **không làm mất nghiệm tối ưu**.

Bài 4.3 (0,5 điểm) — Sudoku: quét tuần tự so với MRV

Cài đặt hai chương trình giải đề Sudoku ở mục E. Hai chương trình dùng CHUNG một thuật toán quay lui, chỉ khác ở cách chọn ô để điền tiếp: bản thứ nhất lấy ô trống đầu tiên gặp phải khi quét từ trái sang phải, từ trên xuống dưới; bản thứ hai dùng heuristic MRV, chọn ô trống có ít ứng viên nhất. Đếm số lời gọi hàm đệ quy của mỗi bản và so sánh.

Đại lượng	Quét tuần tự	MRV
Số lời gọi hàm đệ quy	4209	52
Thời gian chạy	0.003352 giay	0.000895 giay
Lời giải tìm được có giống nhau không?	Có	Có

Python

```
import time

# Ma trận đề bài Sudoku mục E (0 là ô trống)
board = [
    [5, 3, 0, 0, 7, 0, 0, 0, 0],
    [6, 0, 0, 1, 9, 5, 0, 0, 0],
    [0, 9, 8, 0, 0, 0, 0, 6, 0],
    [8, 0, 0, 0, 6, 0, 0, 0, 3],
    [4, 0, 0, 8, 0, 3, 0, 0, 1],
    [7, 0, 0, 0, 2, 0, 0, 0, 6],
    [0, 6, 0, 0, 0, 0, 2, 8, 0],
    [0, 0, 0, 4, 1, 9, 0, 0, 5],
    [0, 0, 0, 0, 8, 0, 0, 7, 9]
]

calls_count = 0

def is_valid(b, r, c, num):
    for i in range(9):
        if b[r][i] == num or b[i][c] == num:
            return False
    br, bc = 3 * (r // 3), 3 * (c // 3)
    for i in range(3):
        for j in range(3):
            if b[br + i][bc + j] == num:
                return False
    return True

def solve_sequential(b):
    global calls_count
    calls_count += 1
    for r in range(9):
        for c in range(9):
            if b[r][c] == 0:
                for num in range(1, 10):
                    if is_valid(b, r, c, num):
                        b[r][c] = num
                        if solve_sequential(b):
                            return True
                        b[r][c] = 0
                return False
```

```
    return True

def solve_mrv(b):
    global calls_count
    calls_count += 1

    best_cell = None
    min_options = 10
    best_nums = []

    for r in range(9):
        for c in range(9):
            if b[r][c] == 0:
                opts = [n for n in range(1, 10) if is_valid(b, r, c, n)]
                if len(opts) < min_options:
                    min_options = len(opts)
                    best_cell = (r, c)
                    best_nums = opts
                    if min_options == 0:
                        return False # Đi vào ngõ cụt
                    if min_options == 1:
                        break
        if min_options == 1:
            break

    if best_cell is None:
        return True # Đã điền kín bảng

    r, c = best_cell
    for num in best_nums:
        b[r][c] = num
        if solve_mrv(b):
            return True
        b[r][c] = 0
    return False

def run_solver(solver_name, solver_func):
    global calls_count, board
    b = [row[:] for row in board]
    calls_count = 0

    start = time.perf_counter()
```

```
    success = solver_func(b)
    end = time.perf_counter()

    print(f"--- {solver_name} ---")
    if success:
        print(f"Thoi gian chay: {end - start:.6f} giay")
        print(f"So loi goi de quy: {calls_count}")
    else:
        print("Khong tim thay loi giai!")

if __name__ == "__main__":
    run_solver("QUET TUAN TU", solve_sequential)
    print()
    run_solver("MRV", solve_mrv)
```

C++

```
#include <iostream>
#include <vector>
#include <string>
#include <chrono>

using namespace std;

// Ma trận đề bài Sudoku mục E (0 là ô trống)
vector<vector<int>> board = {
    {5, 3, 0, 0, 7, 0, 0, 0, 0},
    {6, 0, 0, 1, 9, 5, 0, 0, 0},
    {0, 9, 8, 0, 0, 0, 0, 6, 0},
    {8, 0, 0, 0, 6, 0, 0, 0, 3},
    {4, 0, 0, 8, 0, 3, 0, 0, 1},
    {7, 0, 0, 0, 2, 0, 0, 0, 6},
    {0, 6, 0, 0, 0, 0, 2, 8, 0},
    {0, 0, 0, 4, 1, 9, 0, 0, 5},
    {0, 0, 0, 0, 8, 0, 0, 7, 9}
};

long long calls_count = 0;

bool is_valid(const vector<vector<int>>& b, int r, int c, int num) {
    for (int i = 0; i < 9; i++) {
        if (b[r][i] == num || b[i][c] == num) return false;
    }
    int br = 3 * (r / 3), bc = 3 * (c / 3);
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (b[br + i][bc + j] == num) return false;
        }
    }
    return true;
}

bool solve_sequential(vector<vector<int>>& b) {
    calls_count++;
    for (int r = 0; r < 9; r++) {
        for (int c = 0; c < 9; c++) {
            if (b[r][c] == 0) {
                for (int num = 1; num <= 9; num++) {
                    if (is_valid(b, r, c, num)) {
                        b[r][c] = num;
```

```
                    if (solve_sequential(b)) return true;
                    b[r][c] = 0;
                }
            }
            return false;
        }
    }
    return true;
}

bool solve_mrv(vector<vector<int>>& b) {
    calls_count++;
    int best_r = -1, best_c = -1;
    int min_options = 10;
    vector<int> best_nums;

    for (int r = 0; r < 9; r++) {
        for (int c = 0; c < 9; c++) {
            if (b[r][c] == 0) {
                vector<int> opts;
                for (int num = 1; num <= 9; num++) {
                    if (is_valid(b, r, c, num)) opts.push_back(num);
                }
                if ((int)opts.size() < min_options) {
                    min_options = opts.size();
                    best_r = r;
                    best_c = c;
                    best_nums = opts;
                    if (min_options == 0) return false;
                    if (min_options == 1) break;
                }
            }
        }
        if (min_options == 1) break;
    }

    if (best_r == -1) return true;

    for (int num : best_nums) {
        b[best_r][best_c] = num;
        if (solve_mrv(b)) return true;
        b[best_r][best_c] = 0;
    }
```

```
    return false;
}

void run_solver(string name, bool(*solver)(vector<vector<int>>&)) {
    vector<vector<int>> b = board;
    calls_count = 0;
    auto start = chrono::high_resolution_clock::now();
    bool success = solver(b);
    auto end = chrono::high_resolution_clock::now();
    double time_taken = chrono::duration_cast<chrono::microseconds>(end -
start).count() / 1000000.0;

    cout << "--- " << name << " ---" << endl;
    if (success) {
        cout << "Thoi gian chay: " << time_taken << " giay" << endl;
        cout << "So loi goi de quy: " << calls_count << endl;
    }
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    run_solver("QUET TUAN TU", solve_sequential);
    cout << endl;
    run_solver("MRV", solve_mrv);
    return 0;
}
```

Julia (chỉ số đánh từ 1)

```
# Ma trận đề bài Sudoku mục E (0 là ô trống)
board = [
    5 3 0 0 7 0 0 0 0;
    6 0 0 1 9 5 0 0 0;
    0 9 8 0 0 0 0 6 0;
    8 0 0 0 6 0 0 0 3;
    4 0 0 8 0 3 0 0 1;
    7 0 0 0 2 0 0 0 6;
    0 6 0 0 0 0 2 8 0;
    0 0 0 4 1 9 0 0 5;
    0 0 0 0 8 0 0 7 9
]

calls_count = 0

function is_valid(b::Matrix{Int}, r::Int, c::Int, num::Int)
    for i in 1:9
        if b[r, i] == num || b[i, c] == num
            return false
        end
    end
    br = 3 * div(r - 1, 3) + 1
    bc = 3 * div(c - 1, 3) + 1
    for i in 0:2
        for j in 0:2
            if b[br + i, bc + j] == num
                return false
            end
        end
    end
    return true
end

function solve_sequential!(b::Matrix{Int})
    global calls_count
    calls_count += 1
    for r in 1:9, c in 1:9
        if b[r, c] == 0
            for num in 1:9
                if is_valid(b, r, c, num)
                    b[r, c] = num
                    if solve_sequential!(b) return true end
                    b[r, c] = 0
```

```
            end
        end
        return false
    end
  end
  return true
end

function solve_mrv!(b::Matrix{Int})
  global calls_count
  calls_count += 1

  best_r, best_c = 0, 0
  min_opts = 10
  best_nums = Int[]

  for r in 1:9, c in 1:9
    if b[r, c] == 0
      opts = [n for n in 1:9 if is_valid(b, r, c, n)]
      if length(opts) < min_opts
        min_opts = length(opts)
        best_r, best_c = r, c
        best_nums = opts
        if min_opts == 0 return false end
        if min_opts == 1 break end
      end
    end
  end

  if best_r == 0 return true end

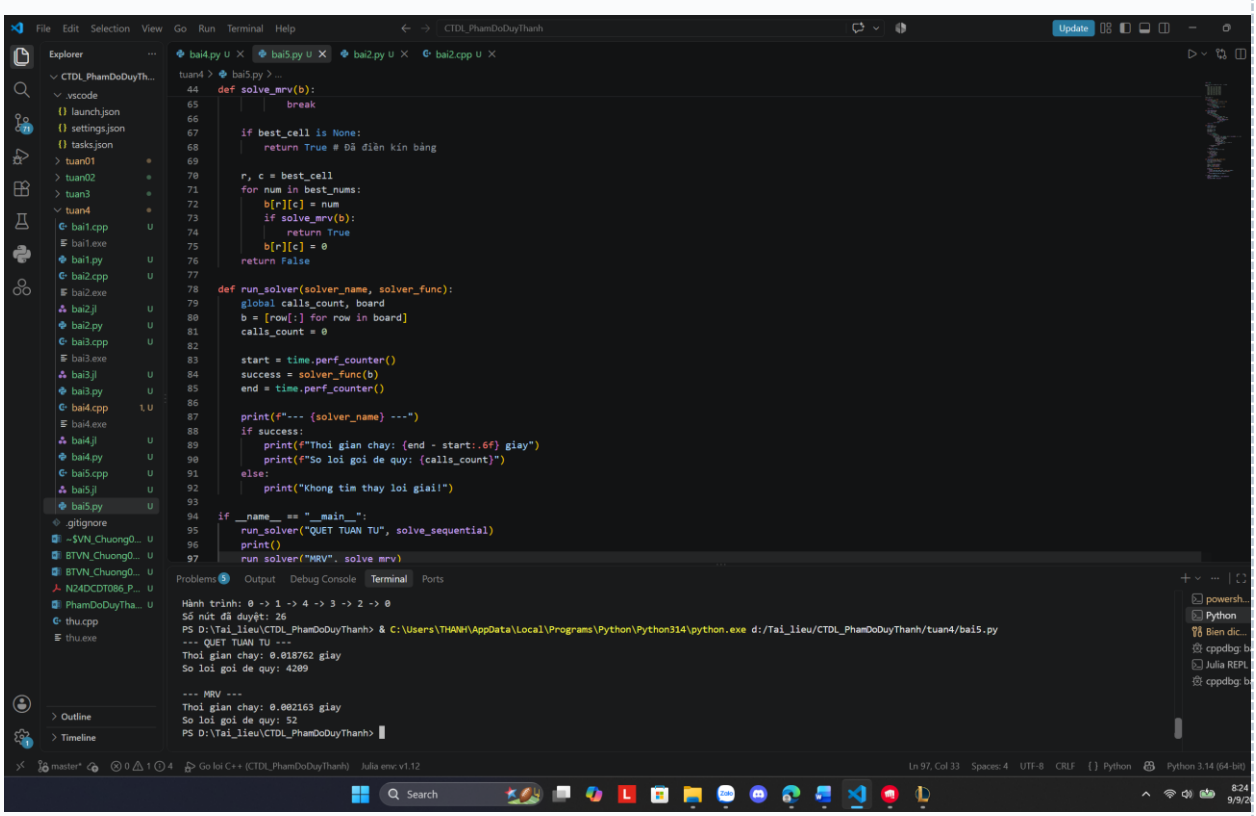
  for num in best_nums
    b[best_r, best_c] = num
    if solve_mrv!(b) return true end
    b[best_r, best_c] = 0
  end
  return false
end

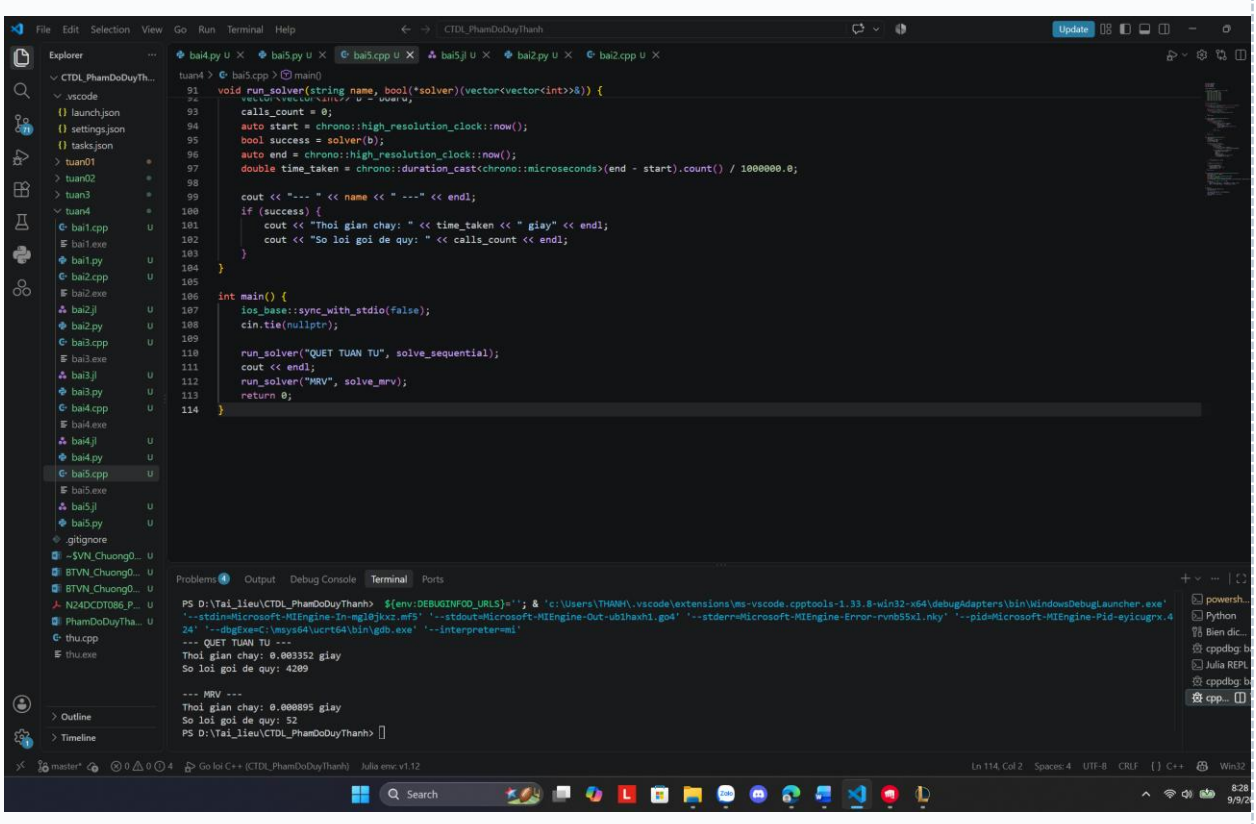
function run_solver(name::String, solver::Function)
  global calls_count
  b = copy(board)
  calls_count = 0
```

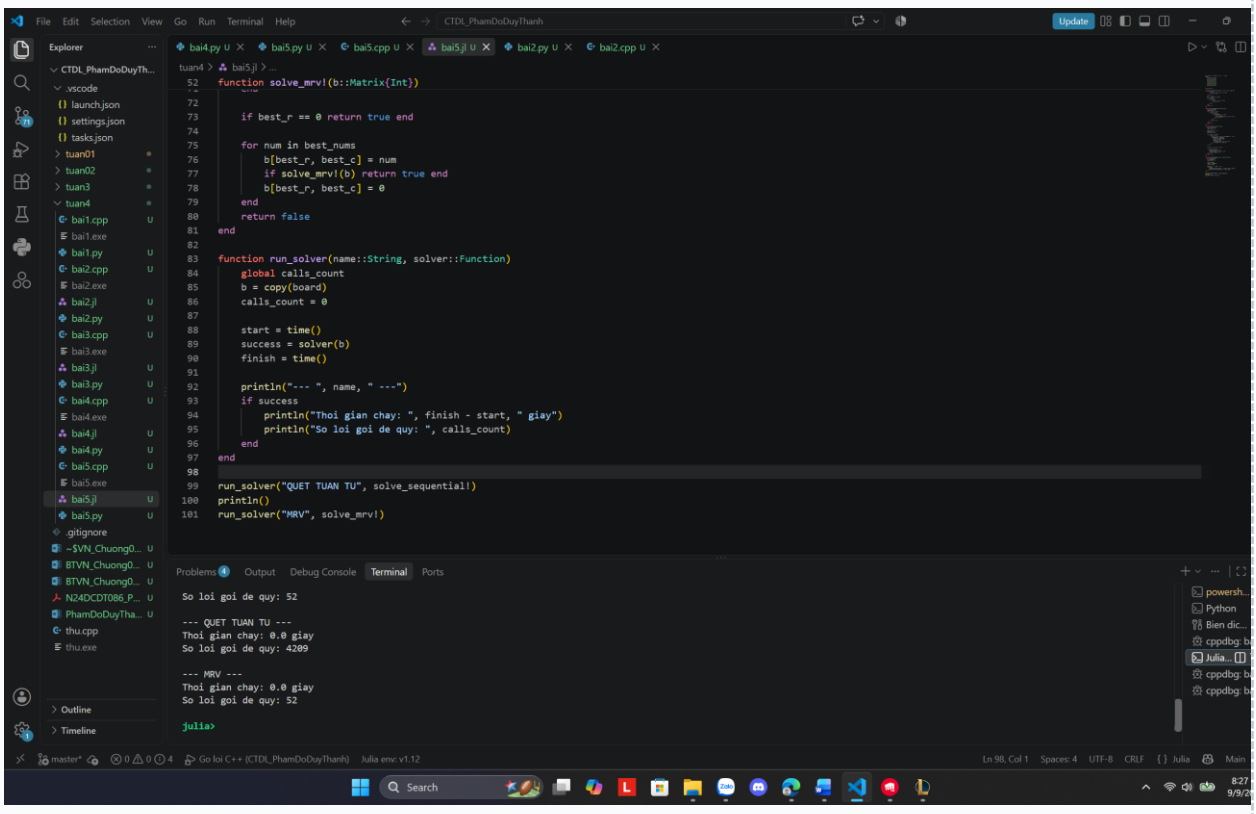
```
    start = time()
    success = solver(b)
    finish = time()

    println("--- ", name, " ---")
    if success
        println("Thoi gian chay: ", finish - start, " giay")
        println("So loi goi de quy: ", calls_count)
    end
end

run_solver("QUET TUAN TU", solve_sequential!)
println()
run_solver("MRV", solve_mrv!)
```


def solve_mrv(b):
break
if best_cell is None:
return True # Đã điền kín bảng
r, c = best_cell
for num in best_nums:
b[r][c] = num
if solve_mrv(b):
return True
b[r][c] = 0
return False
def run_solver(solver_name, solver_func):
global calls_count, board
b = [row[:] for row in board]
calls_count = 0
start = time.perf_counter()
success = solver_func(b)
end = time.perf_counter()
print(f"--- {solver_name} ---")
if success:
print(f"Thoi gian chay: {end - start:.6f} giay")
print(f"So loi goi de quy: {calls_count}")
else:
print("Khong tim thay loi giai!")
if __name__ == "__main__":
run_solver("QUET TUAN TU", solve_sequential)
print()
run_solver("MRV", solve_mrv)
Hành trình: 0 -> 1 -> 4 -> 3 -> 2 -> 0
Số nút đã duyệt: 26
PS D:\Tai_lieu\CTDL_PhamDoDuyThanh> & C:\Users\THANH\AppData\Local\Programs\Python\Python314\python.exe d:/Tai_lieu/CTDL_PhamDoDuyThanh/tuan4/bai5.py
--- QUET TUAN TU ---
Thoi gian chay: 0.018762 giay
So loi goi de quy: 4209
--- MRV ---
Thoi gian chay: 0.002163 giay
So loi goi de quy: 52
PS D:\Tai_lieu\CTDL_PhamDoDuyThanh>


void run_solver(string name, bool(*solver)(vector<vector<int>>&)) {
calls_count = 0;
auto start = chrono::high_resolution_clock::now();
bool success = solver(b);
auto end = chrono::high_resolution_clock::now();
double time_taken = chrono::duration_cast<chrono::microseconds>(end - start).count() / 1000000.0;
cout << "--- " << name << " ---" << endl;
if (success) {
cout << "Thoi gian chay: " << time_taken << " giay" << endl;
cout << "So loi goi de quy: " << calls_count << endl;
}
}
int main() {
ios_base::sync_with_stdio(false);
cin.tie(nullptr);
run_solver("QUET TUAN TU", solve_sequential);
cout << endl;
run_solver("MRV", solve_mrv);
return 0;
}
--- QUET TUAN TU ---
Thoi gian chay: 0.003352 giay
So loi goi de quy: 4209
--- MRV ---
Thoi gian chay: 0.000895 giay
So loi goi de quy: 52
PS D:\Tai_lieu\CTDL_PhamDoDuyThanh>



Câu hỏi phải trả lời, đây mới là phần cho điểm

Hai bản dùng chung một thuật toán quay lui, chỉ khác thứ tự chọn ô. Sau khi chạy xong, hãy giải thích **vì sao chỉ đổi thứ tự chọn ô mà số lời gọi lại thay đổi nhiều đến thế**, và trả lời: heuristic MRV có khả năng làm thuật toán bỏ sót lời giải hoặc cho ra lời giải sai không? Vì sao?

1. Vì sao chỉ đổi thứ tự chọn ô mà số lời gọi đệ quy lại thay đổi nhiều đến thế?

- **Bản chất vấn đề:** Thuật toán quay lui duyệt theo chiều sâu (DFS). Khi chọn một ô trống để điền, số lượng giá trị hợp lệ của ô đó quyết định số nhánh rẽ của cây tìm kiếm.
- **Với cách quét tuần tự:** Thuật toán chọn ô trống theo thứ tự cố định, dù ô đó có thể có 4–5 giá trị hợp lệ. Điều này làm cây tìm kiếm phân nhánh rộng. Nếu các nhánh dẫn đến bế tắc ở những ô phía sau, thuật toán chỉ phát hiện sai sau nhiều bước, dẫn đến phải duyệt qua rất nhiều trường hợp sai và tốn tới **4209** lời gọi đệ quy.
- **Với heuristic MRV:** Thuật toán ưu tiên chọn ô có **ít giá trị hợp lệ nhất** để điền trước.
    - Nếu một ô có **0 ứng viên**, nhánh được xác định là sai ngay và quay lui sớm.
    - Nếu một ô chỉ có **1 ứng viên**, thuật toán có thể điền ngay mà không tạo thêm nhánh rẽ.
- Nhờ phát hiện các nhánh không khả thi sớm hơn, số lời gọi đệ quy giảm đáng kể từ **4209** xuống chỉ còn **52** lần.

2. Heuristic MRV có khả năng làm thuật toán bỏ sót lời giải hoặc cho ra lời giải sai không? Vì sao?

Không, nếu MRV được áp dụng đúng và thuật toán vẫn thử đầy đủ các ứng viên hợp lệ.

- **Không bỏ sót nghiệm:** MRV chỉ thay đổi **thứ tự chọn ô**, không loại bỏ các giá trị hợp lệ của bài toán. Với mỗi ô được chọn, thuật toán vẫn thử các ứng viên hợp lệ và quay lui đầy đủ. Vì vậy, nếu Sudoku có nghiệm thì thuật toán vẫn có thể tìm ra nghiệm.
- **Không cho lời giải sai:** Trước khi điền một số, chương trình vẫn kiểm tra đầy đủ ba điều kiện của Sudoku: số đó không được trùng trong **hàng, cột và khối 3 x 3**. Vì vậy, khi bảng được điền đầy đủ, nghiệm thu được vẫn thỏa mãn các luật của Sudoku.